The Journal of Systems and Software 225 (2025) 112326

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

UVL: Feature modelling with the Universal Variability Language ☆

David Benavides [a,*], Chico Sundermann [b], Kevin Feichtinger [c], José A. Galindo [a], Rick Rabiser [d], Thomas Thüm [e]

[a] Department of Computer Languages and Systems, I3US, Universidad de Sevilla, Av. Reina Mercedes, Seville, 41012, Spain
[b] Institute of Software Engineering and Programming Languages, University of Ulm, Albert-Einstein-Allee 11, Ulm, 89069, Germany
[c] CRC 1608, KASTEL – Dependability of Software-intensive Systems, Karlsruhe Institute of Technology, Am Fasanengarten 5, Karlsruhe, 76131, Germany
[d] Christian Doppler Laboratory VaSiCS, LIT CPS Lab, Johannes Kepler University Linz, Altenberger Straße 69, Linz, 4040, Austria
[e] Institute of Software Engineering and Automotive Informatics, TU Braunschweig, Mühlenpfordtstr. 23, Braunschweig, 38106, Germany

ARTICLE INFO

Dataset link: https://github.com/Universal-Variability-Language

Keywords:
Feature model
Software product lines
Variability

ABSTRACT

Feature modelling is a cornerstone of software product line engineering, providing a means to represent software variability through features and their relationships. Since its inception in 1990, feature modelling has evolved through various extensions, and after three decades of development, there is a growing consensus on the need for a standardised feature modelling language. Despite multiple endeavours to standardise variability modelling and the creation of various textual languages, researchers and practitioners continue to use their own approaches, impeding effective model sharing. In 2018, a collaborative initiative was launched by a group of researchers to develop a novel textual language for representing feature models. This paper introduces the outcome of this effort: the Universal Variability Language (UVL), which is designed to be human-readable and serves as a pivot language for diverse software engineering tools. The development of UVL drew upon community feedback and leveraged established literature in the field of variability modelling. The language is structured into three levels – Boolean, Arithmetic, and Type – and allows for language extensions to introduce additional constructs enhancing its expressiveness. UVL is integrated into various existing software tools, such as FeatureIDE and flamapy, and is maintained by a consortium of institutions. All tools that support the language are released in an open-source format, complemented by dedicated parser implementations for Python and Java. Beyond academia, UVL has found adoption within a range of institutions and companies. It is envisaged that UVL will become the language of choice in the future for a multitude of purposes, including knowledge sharing, educational instruction, and tool integration and interoperability. We envision UVL as a pivotal solution, addressing the limitations of prior attempts and fostering collaboration and innovation in the domain of software product line engineering.

1. Introduction

Feature modelling (Kang et al., 1990), a crucial component of software product line engineering, is one of the most used approaches for representing software variability through the abstraction of features and their relationships (Apel et al., 2013). A feature is defined as an increment in product functionality (Batory, 2005). A software product line is modelled using a *Feature Model* (FM) where features are arranged in a tree-like structure with additional cross-tree constraints. FMs with thousands of features are reported in the literature (Berger et al., 2013; Kübler et al., 2010; Oh et al., 2023; Sundermann et al., 2023a). FMs are represented using feature diagrams but can also be represented using different textual notations. Textual notations for FMs range from XML–based to tool-specific ones (Beek et al., 2019). Over the past thirty years, the evolution of feature modelling has given rise to diverse extensions and representations (Eichelberger and Schmid, 2015). However, the absence of a standardised language has impeded effective model sharing among researchers and practitioners, hindering progress in the field.

The year 2018 marked a turning point as a collaborative initiative emerged intending to address the standardisation challenge. This initiative brought together a group of researchers from different universities and research centres under the umbrella of the MODEVAR[1] workshop

☆ Editor: Laurence Duchien.

* Corresponding author.
E-mail addresses: benavides@us.es (D. Benavides), chico.sundermann@uni-ulm.de (C. Sundermann), kevin.feichtinger@kit.edu (K. Feichtinger), jagalindo@us.es (J.A. Galindo), rick.rabiser@jku.at (R. Rabiser), thomas.thuem@tu-braunschweig.de (T. Thüm).

[1] https://modevar.github.io/

https://doi.org/10.1016/j.jss.2024.112326
Received 15 March 2024; Received in revised form 1 October 2024; Accepted 18 December 2024
Available online 31 January 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

series. This effort was dedicated to crafting a new textual and simple language for FMs (Sundermann et al., 2021a). The outcome of this collective effort is the *Universal Variability Language* (UVL), a solution designed to be both human-readable and a pivot language for a variety of software engineering tools.

UVL's development was not only informed by community feedback but also based on established literature in the field of variability modelling. The language is meticulously structured into three levels –*Boolean*, *Arithmetic*, and *Type*– allowing for a representation of different features and varying types of relationships. Furthermore, UVL embraces extensibility, permitting the introduction of additional constructs to enhance its expressiveness and accommodate diverse modelling needs.

UVL is integrated into existing variability modelling tools, such as FeatureIDE (Kästner et al., 2009) and flamapy (Galindo et al., 2023). All the tools that support the language are available in an open-source format, complemented by dedicated parser implementations for Python and Java using ANTLR (Parr, 2013),[2] which allows designing parsers for other languages. This openness paired with a structured process to involve the community encourages transparency, collaboration, and wider adoption.

We envision an impact of UVL beyond academia, with institutions and companies recognising its potential. As an example, an importer for UVL models was already integrated in a commercial variant management tool (Romano et al., 2022). UVL could be used as the language of choice for a myriad of applications, including knowledge sharing, educational instruction, and seamless tool integration. The broad vision for UVL is to overcome the limitations of previous standardisation attempts, such as the Common Variability Language (CVL) (Haugen et al., 2012) or ISO-26558 (ISO/IEC 26558:2017(en), 2017). CVL (Haugen et al., 2012) eventually and unfortunately failed to become a standard due to legal reasons (Rabiser, 2019) and ISO-26558 (ISO/IEC 26558:2017(en), 2017) did not reach the community and industry. However, as UVL is community driven, we envision UVL to foster collaboration and innovation within the realm of software product line engineering.

In this paper, we delve into the development, features, and applications of UVL, offering a comprehensive exploration of its significance in the evolving landscape of variability modelling. The contributions of the paper are as follows:

- A tutorial presentation of UVL with a stable version of the language (Section 4) validated by different rounds of participation by the community.
- An extensible language design that provides expressive language features while preserving simplicity with a core language divided into three major levels and an option to decompose large feature models.
- A formal textual syntax and semantics of UVL (Section 5).
- An open source implementation[3] of the language with parsers for Python and Java using ANTLR that allows supporting new general languages such as JavaScript or C# in the future (Section 6).
- A report of our experiences regarding the feasibility of the language based on an interactive and participated process with the community (Section 3) as well as the integration of UVL with different tools (Section 6).

Regarding novelty since previous publications (Feichtinger et al., 2021; Munoz et al., 2023; Romano et al., 2022; Sundermann et al., 2021a, 2023b), different changes have been introduced and no stable version of UVL was presented so far. The formal syntax of this stable version of UVL as well as the parser implementation supporting Java and Python are new. Furthermore, this work includes the first formal specification and discussion on the semantics of UVL.

The remainder of the paper is structured as follows. Section 2 introduced the necessary background on FMs. We outline the development process and the design goals of UVL in Section 3. After that, we introduce the UVL in a tutorial-like manner and discuss its syntax and semantics in Sections 4 and 5, respectively. We provide an overview of the current UVL implementation and existing tools integrating UVL in Section 6. We then discuss challenges and next steps regarding further adoption of UVL in general and in industry in particular in Section 7. Section 8 concludes the paper.

2. Feature models

The term *Feature Model* (FM) was coined by Kang et al. in the well-known FODA report in 1990 (Kang et al., 1990). Since then, feature modelling has been one of the main topics of research in software product lines (Galindo et al., 2019; Felfernig et al., 2024). There are different FM dialects (Schobbens et al., 2007), each with different types of features or relationships, but also with different textual and graphical notations. In the following, we review the most used notations for those languages to pave the way for the presentation of UVL. In general, there is no FM language that can be used in all scenarios and adaptations are often done for concrete domains (Alférez et al., 2019).

An FM is a representation of all possible configurations of a software product line (Felfernig et al., 2024). Given n features, with no restrictions in the combinations of them, 2^n is the number of all potential configurations. With a small n in terms of hundreds, the number of configurations is already very big. An FM restricts this number using feature relationships that represent the constraints of the application domain. FMs are also used in other domains than software product lines such as video encoding (Alférez et al., 2019), biological information (Cashman et al., 2019) or exam options (Le et al., 2021) just to mention a few examples. One of the most used examples in the community is the Linux kernel FM, which has thousands of modules and configuration options (Thüm, 2020). Furthermore, large FMs from other domains, such as automotive, with thousands or even tens of thousands of features were reported in the literature (Kübler et al., 2010; Knüppel et al., 2017; Sundermann et al., 2023a; Berger et al., 2013). Still, there might be even larger FMs used in practice as FMs from industry are typically not made available.

Fig. 1 shows our running example of the FM for a fictitious `eShop` product line. An FM is composed of a hierarchically arranged set of features (a.k.a. *feature diagram* or *feature tree*) and a set of cross-tree constraints. FMs that do not have cross-tree constraints can exist, but they are not very common. Relationships among features can be of two different types (Benavides et al., 2010): *i*) relationships between a parent feature and its child features; *ii*) cross–tree constraints that are typically inclusion or exclusion statements or more complex constraints in the form of arbitrary (propositional) formulae.

There are several FM dialects (Schobbens et al., 2007). In this section, we will revisit the most used relationships in the literature and also mention some of the extensions proposed. In basic FMs, the following relationships among features are defined (Benavides et al., 2010):

- **Mandatory**. A child feature has a *mandatory* relationship with its parent if the child is part of all configurations in which its parent feature is included, e.g., any configuration of an `eShop` has to have a `Catalogue`.
- **Optional**. A child feature has an *optional* relationship with its parent if the child can optionally be part of all configurations in which its parent feature is included, e.g., a configuration of an `eShop` can optionally have `SEO` support.

[2] https://www.antlr.org/
[3] https://github.com/Universal-Variability-Language

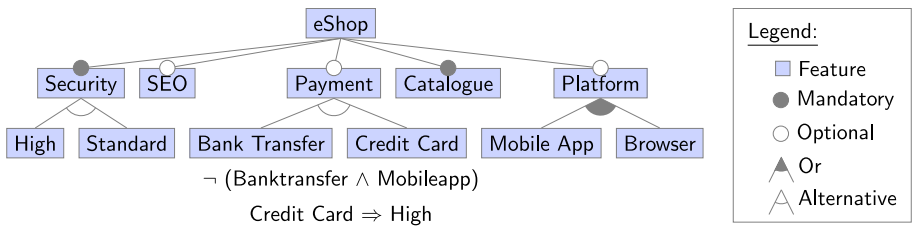

Fig. 1. Running example of the online shop case study (Rodas-Silva et al., 2019).

- **Alternative**. Child features have an *alternative* relationship with their parent if exactly one of them can be part of a configuration if the parent feature is included. In the example, the *Payment* of the `eShop` must be either `Banktransfer` or `Credit card` (but not both in the same configuration).
- **Or**. Child features have an *or* relationship with their parent if one or more of them can be part of the configuration if the parent feature is included. In Fig. 1, whenever `Platform` is selected, `Mobileapp`, `Browser` or any combination thereof including at least one of these two features can be selected.

Note that always a child feature can only be part of a configuration if its parent feature is part of the configuration. Additionally, the root feature is included in all the configurations of the product line. An FM can also contain cross-tree constraints between features — basic ones are the following:

- **Requires**. If a feature *A requires* a feature *B*, the inclusion of *A* in a configuration implies the inclusion of B in such a configuration. In the example, an `eShop` including `Credit card` must include `High` security support.
- **Excludes**. If a feature *A excludes* a feature *B*, both cannot be included in the same configuration, i.e., there is a feature exclusion. The `Banktransfer` feature cannot be combined with a `Mobileapp`, i.e., these two features are incompatible.

More complex cross-tree relationships are often used allowing constraints in the form of generic propositional formulas, e.g., "`A` and not `B` implies `C`" (Batory, 2005). In some cases, there is a distinction between *concrete* and *abstract* features (Thüm et al., 2011). Concrete features have a mapping with domain implementation artefacts in the solution space (Apel et al., 2013), while abstract features are used for organisation purposes and do not have any direct mapping to any artefact in the solution space. Often, only leaves of the tree are concrete features and all the other intermediate features are abstract (Batory, 2020a).

2.1. Feature model extensions

There are different ways to extend FMs with different constructs. The most well-known families of extensions are *cardinality–based* and *attribute–based* FMs. These extensions include a discussion that has been going on in the community over the years: what are the semantics of feature cardinalities, cloning, or attributes? (Classen et al., 2011; Czarnecki et al., 2005b; El-Sharkawy et al., 2015; Riebisch et al., 2002; Rothberg et al., 2016) In this section, we do not repeat such discussions in detail. In following sections when UVL is presented, more details on how those discussions are taken into account will be reported.

Cardinality–based FMs introduce two relationships that resemble those of the *Unified Modelling Language* (UML) with multiplicities in class diagrams — see Czarnecki et al. (2005), Riebisch et al. (2002). The relationships introduced in cardinality–based feature modelling are the following (Benavides et al., 2010):

- **Feature cardinality.** A feature cardinality is a sequence of intervals $[n..m]$ with n as lower bound and m as upper bound ($n \leq m$). Feature cardinalities are also known as feature clones. The intervals describe the number of instances of the feature that can be part of a configuration. This relationship may be used as a generalisation of the original mandatory ($[1,1]$) and optional ($[0,1]$) relationships defined in classical FMs described previously. Cloning a feature means having different instances of the same feature several times in a configuration.
- **Group cardinality.** A group cardinality is an interval $\langle n..m \rangle$, with n being the lower and m the upper bound ($n \leq m$) limiting the number of child features that can be included in a configuration when the parent feature is selected (remember that if the parent is not included in a configuration, none of its children are included). An alternative relationship is equivalent to a $\langle 1..1 \rangle$ group cardinality. An or-relationship is equivalent to $\langle 1..N \rangle$, being N the number of features in the relationship.

Attribute–based FMs. In certain situations, FMs include additional information about the features. For example, the cost or memory consumption of a particular feature in an `eShop` configuration. Such information can be included using *feature attributes*, which are designed for this specific purpose. When FMs are expanded by including additional information in the form of attributes, they are referred to as *extended, advanced, or attribute-based FMs* (Alférez et al., 2019). Most proposals of attribute-based FMs agree that an attribute should consist at least of a *name*, a *type*, a *domain* and a *value*.

3. Development process and design goals of UVL

We first describe how UVL was developed in a participatory effort in the SPL community and then summarise its design goals.

3.1. Participatory development process

UVL is the result of a community effort that started in 2018 as depicted in Fig. 2. The idea began with an informal meeting at SPLC 2018 with around twenty key researchers from the SPL community. After a brainstorming session, we agreed on several action points. Among those, it was decided to run a workshop (MODEVAR[4]) to be "an interactive event where all participants shall share knowledge about how to build up a simple feature model language that all the community can agree on".

In the 2019 edition (Benavides et al., 2019), the main outputs were a revisited literature review on textual variability modelling languages (Beek et al., 2019), the proposal of having different language levels (Thüm et al., 2019) and a set of fourteen usage scenarios of the language (Berger and Collet, 2019) described by examples. Those scenarios were the result of a systematic process, where members of the community gave original descriptions, which received feedback via

[4] https://modevar.github.io/

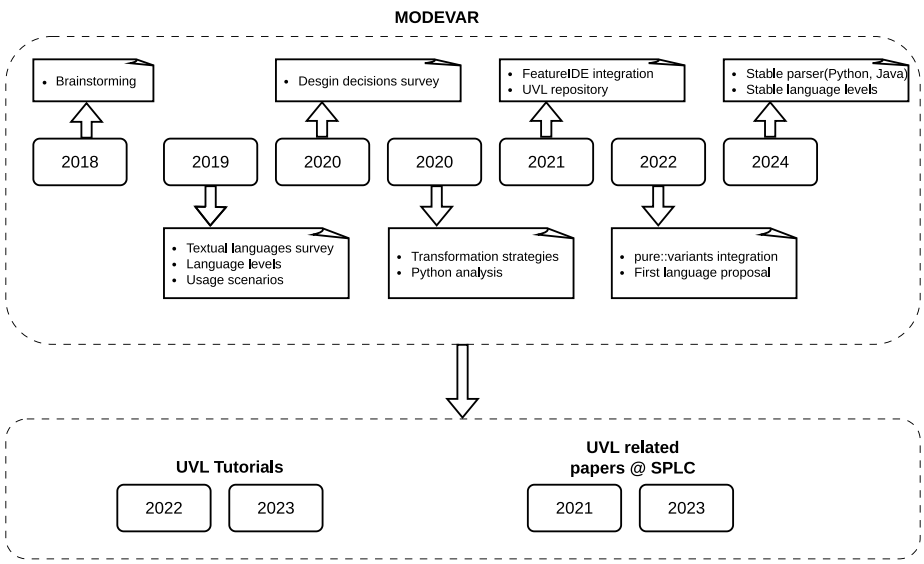

Fig. 2. UVL development process since 2018.

a survey and expert feedback. The survey, the language levels, and the usage scenarios were used for the next steps in the process.

During the 2020 VaMoS event, a survey (results later published at SPLC 2021 (Sundermann et al., 2021a)) aimed at informing the language's design decisions was performed. This survey comprised a questionnaire administered to 20 workshop attendees. In the initial part of the questionnaire, participants shared their preferences concerning the gathered structural attributes of the language. Subsequently, in the latter part, attendees deliberated on which language features should be incorporated based on their considerations. Throughout the questionnaire, participants collaborated in pairs to deliberate on their viewpoints and offer more meticulously considered responses.

The workshop was run again in 2020 at SPLC (Acher et al., 2020) (online due to the pandemic). Transforming different variability models is a challenge, in Feichtinger and Rabiser (2020a) usage scenarios, required capabilities and challenges for an approach for semi-automatically transforming variability models were presented. One of the conclusions was that a pivotal common language can help to transform variability artefacts and underlines the necessity of UVL in this sense as a pivotal language for transformations. In addition, a new tool based on Python to analyse FMs was presented (Galindo and Benavides, 2020) with the potential of including UVL as variability language (see Section 6.2.3).

In 2021 (Thüm et al., 2021), two concrete integrations of UVL in different tools were presented. First, an integration of one of the previous versions of UVL in FeatureIDE showed the feasibility of the language (Sundermann et al., 2021b). Second, a prototype of a repository to share UVL models was presented (Romero et al., 2021). This way, some of the objectives of the language started to materialise: tools integration and knowledge sharing (see Section 6).

In 2022 (Horcas et al., 2022b), another integration, in this case with a commercial variant management tool was presented (Romano et al., 2022). Concretely, UVL was integrated with pure::variants (pure::systems, 2017), one of the most well-known commercial tools in the software product line engineering area. In addition, a first tutorial on a previous version of UVL was given.

During these years, some tools integrated different versions of UVL producing their own parsers (Galindo et al., 2023; Loth et al., 2023; Heß et al., 2022a). In 2023, there was an implementation effort to produce a common stable parser of UVL with support for Python and Java. This parser was briefly introduced during the MODEVAR 2024 edition at VaMoS 2024 and it is one of the contributions of this paper (cf. sections 4 and 6). Additionally, further developments around UVL and its expressiveness were presented (Agarwal et al., 2024; Heß et al., 2024; Romero-Organvidez et al., 2024a).

In 2024 a second MODEVAR edition took place (Feichtinger and Galasso-Carbonnel, 2024). This time the focus of the community turned towards the adoption of UVL in industry. For that, potential challenges (Rabiser, 2024) and necessary extensions (Fadhlillah and Rabiser, 2024) for UVL were discussed with a representative of pure::variants (pure::systems, 2017). Additionally, a generator for UVL models in arbitrary size and complexity was introduced, which facilities scalability analysis (Sundermann et al., 2024).

Although MODEVAR has been the meeting point of researchers and practitioners with interest in the development of a simple, common textual feature modelling language, the outputs and discussions of the workshop served to produce other artefacts outside of the workshop (Krieter et al., 2023; Sundermann et al., 2022). Concretely, there were two tutorials at SPLC 2022 and 2023 presenting the advances of UVL as well as analysis and transformation capabilities. Also, there were two major papers at SPLC 2021 and 2023 presenting a first version of the language (Sundermann et al., 2021a) and some transformation and analysis capabilities (Sundermann et al., 2023b).

In summary, one unique selling point of UVL is the community-driven design of the language (Sundermann et al., 2021a). With various surveys and discussions with experts of the community, different authors derived requirements for the design of a widely adopted variability language (Beek et al., 2019; Benavides et al., 2019; Berger and Collet, 2019; Sundermann et al., 2021a; Thüm et al., 2019). In the following, we present derived requirements that influenced the language design and how we address these in UVL.

3.2. Design goals

Designing a language is difficult (Wąsowski and Berger, 2023). With the participatory process described in the previous section, we mitigated the possibility of having a language that was not accepted by the community. With the inputs of the workshops and working sessions, we defined several design goals that are summarised as follows:

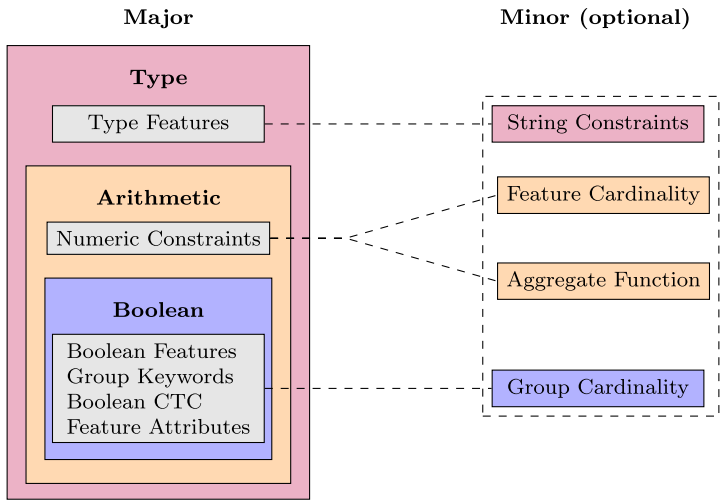

Fig. 3. Language level hierarchy in UVL.

Simplicity. In general, UVL should be simple to use. For simplicity, we consider two dimensions: (1) UVL should be easy to use, understand (with simple constructs), learn and comprehend (facilitating the comprehension of the variability in hand) for humans (Berger and Collet, 2019) and (2) it should not require too much effort to integrate UVL in variability modelling tools. For human understandability and comprehension, UVL should use concepts familiar to users. As potential users, we consider people working in the computer science field and/or using variability modelling. Hence, we aim to use concepts from programming languages, modelling in computer science (e.g., grammars or meta-models), and existing variability languages. For easier integration, we consider the following requirements for UVL. First, the core language should be simple so that developers do not have to integrate various complex constructs. Second, the language should reuse existing concepts from other variability languages, e.g., common keywords like *alternative*. Third, the core language should be simple to analyse with conventional analysis tools used in the domain, such as SAT (Czarnecki and Wąsowski, 2007; Mendonça et al., 2009b; Zhang et al., 2004), BDD (Heradio et al., 2022) or #SAT solvers (Kübler et al., 2010; Liang et al., 2015; Sundermann et al., 2023a).

Information hiding. In practice, it often makes sense to only work on small subparts of a variability model. First, large variability models are typically hard to oversee Acher et al. (2009). Second, different stakeholders commonly do not work on the entire variability model but specific parts (Acher et al., 2009). Hence, UVL should have a mechanism to support focusing on a subset of interest.

Expressiveness. To be widely applicable, one of UVL's goals is to cover many practical use cases. First, users of UVL should be able to specify constraints as needed to describe the set of valid configurations, which may include propositional logic, constraints over numeric values, or even reasoning about content of strings (Beek et al., 2019). Second, UVL should be able to describe constructs used in available feature modelling tools (Acher et al., 2013; Galindo et al., 2023; Meinicke et al., 2017; Mendonça et al., 2009a).

Extensibility. A higher expressiveness conflicts with the goal of simplicity (Thüm et al., 2019), as more and potentially more complex language constructs need to be supported. As a compromise in UVL, we aim to have an extensible language design with a simple core language that can be easily adopted and extensions that introduce more expressiveness. Here, we use the concept of language levels (Thüm et al., 2019) that encapsulate different language constructs and extend the UVL core language.

Exchange. Models of a common variability language should be exchangeable between different tools (Berger and Collet, 2019). For simplifying exchange, we consider two aspects. First, available tool support (e.g., for parsing) should be reusable for different users of UVL. Second, there should be a mechanism to exchange UVL models between tools that support different levels of expressiveness.

4. The Universal Variability Language (UVL)

In this section, we illustrate how to specify variability models with UVL[5] using our running example. The design of UVL consists of a simple base language with several language extensions, which we call language levels (cf. Section 5). Here, we start with a simple version and extend it iteratively to showcase more expressive UVL language levels. For a formal description on the language, we refer to Section 5.

4.1. Language levels

In UVL, we use *language levels* to tackle expressiveness and extensibility while preserving a simple core language. The idea is that users of UVL can limit their models to specific language constructs. If a tool only supports very simple constructs, higher language levels can be forbidden. If more expressiveness is needed, additional language levels can be enabled.

Fig. 3 shows the language levels currently available in UVL. Each language level encapsulates certain language constructs. We distinguish between *major* and *minor* language levels. The major levels have a hierarchical order. The *Boolean*-level is the *core* language of UVL. The *Arithmetic*-level fully includes the major Boolean level and extends it with numeric constraints over feature attributes. The *Type*-level extends both with typed features, such as string or numeric features. The goal of these levels is to separate the language according to reasoning engines that could be used to reason about them. For instance, the *Boolean*-level can be simply encoded as a SAT problem. *Minor* language levels are optional extensions of the major levels. The idea is to separate constructs that can be analysed with the same reasoning engine but may require further handling or are not always supported by available tools. They are not automatically included in higher language levels.

[5] We discussed different name alternatives for the proposed language and we decide to use UVL because the intention is to make it an *Universal* language used by many stakeholders in the variability modelling community.

Listing 1: UVL Running Example: Core

```
features
    eShop
        mandatory // select all
            Security
                alternative // select exactly one
                    High {Price 100}
                    Standard {Price 50}
            Catalogue
        optional
            SEO
            Payment
                alternative
                    "Bank Transfer" {Price 10, SEPA true}
                    "Credit Card" {Price 20}
            Platform
                or // select at least one
                    "Mobile App"
                    Browser {URL 'www.uvleshop.org'}

constraints
    !("Bank Transfer" & "Mobile App")
    "Credit Card" => High
```

Group Cardinality extends the boolean level with cardinality group relationships which enable selecting [n..m] features from the group. *Feature Cardinality* and *Aggregate Functions* are optional extensions of the arithmetic level. They enable (1) selecting a feature multiple times and (2) aggregates, such as sums, over numerical attributes, respectively. *String Constraints* add constraints to compare strings and lengths of strings. In the following, we showcase the different language levels by extending our base example shown in Listing 1.

4.2. Boolean level

Listing 1 shows our running example from Fig. 1 in UVL syntax. The UVL model consists of two main parts: the feature tree and the cross-tree constraints. The tree hierarchy is represented using indentation. Keywords are used to specify the parent–child relationship. As in Fig. 1, `eShop` has two mandatory and three optional child features. Furthermore, exactly one `Security` option and exactly one `Payment` option can be selected as denoted by the *alternative*-keyword. For the feature `Platform`, the *or*-group denotes that at least one platform can be selected. The cross-tree constraints are used to impose further limitations on the features. For instance, `Bank Transfer` and `Mobile App` cannot be included in the same configuration, i.e., they are incompatible features. Also, a `Credit Card` requires a `High` security level. For the core language, the constraints are limited to propositional logic.

In addition to feature dependencies, the UVL model contains some attributes that provide information on the respective features. In our model, we have a number attribute (price), a Boolean attribute (SEPA), and a string attribute (URL). In the core language of UVL, attributes can only be used for storing information about features that do not influence the validity of configurations. Constraints over attributes are excluded in the core language, since the reasoning is considerably more complex and not straightforward to encode for many automated reasoning engines, such as SAT solvers. Still, attributes are relevant to (1) store tool-specific information, (2) attach general information to features, and (3) can be used to compute metrics for configurations based on user selections, such as a price.

For further information, comments can be added either single line with `//` or multiple lines with `/* <comment> */`. All comments are discarded during the parsing process.

Listing 2 shows an adaptation of the previous `eShop` but using now the cardinality capacity for a feature group. Another change is the *include* at the very top of the listing. The include mechanism allows users to specify explicitly which language constructs are supported. This can be used for (1) providing information on the contents of the language level and (2) ensure that users do not introduce constructs that are not supported by the tool using UVL. In the latter case, the UVL parser should provide information on the mismatch of declared and used levels. By default, i.e., when no language levels are specified in includes, all language levels are included. Each construct in the initial `eShop` Listing 1 is part of the core language. Group cardinality is a *minor* level of the Boolean (i.e., core) language level. Including the minor level group-cardinality automatically includes its major level Boolean. In Section 4.3 and Section 4.4, we illustrate the other two major language levels in UVL, namely *Arithmetic* and *Type*, using our running example.

Listing 2: UVL Running Example: Group Cardinality

```
include
    Boolean.group-cardinality

features
    ...
        Platform
            [2..3]
                "Desktop App"
                "Mobile App"
                Browser {URL 'www.uvleshop.org'}
    ...
```

Group cardinality. In addition to the specification of included language levels, there are changes in the feature tree. Now, in Listing 2 the customer has three `Platform` options to choose from. In addition, the group-type changed to a *group cardinality*. The group denotes that the customer needs to select between two and three ([2..3]) platform features instead of at least one.

4.3. Arithmetic level

In this section, we extend our FM with constructs from the *Arithmetic*-level and its minor levels. Listing 3 further enriches our `eShop` with an *arithmetic constraint* over the price attribute. The constraint denotes that the overall sum in price of all selected features should be smaller than 200. With the *Arithmetic*-level, the following operators are supported: +, -, *, /, ==, <, >, <=, and >=. The minor level *aggregate-function* also introduces sum() and avg().

Listing 3: UVL Running Example: Arithmetic

```
include
    Boolean.group-cardinality
    Arithmetic.aggregate-function

features
    ...

constraints
    !("Bank Transfer" & "Mobile App")
    "Credit Card" => High
    sum(Price) < 200
```

Feature cardinality. In Listing 4, we introduce *feature cardinality*, which is a minor level of the *Arithmetic*-level. In our example, the user can decide to have between one and five `Catalogue` features as denoted by *cardinality* [1..5]. A customer may select varying catalogues for different markets, e.g., Europe and North America. Note that each selected `Catalogue` would increase the overall price by 30.

Listing 4: UVL Running Example: Feature Cardinality

```
include
   Boolean.group-cardinality
   Arithmetic.aggregate-function
   Arithmetic.feature-cardinality

features
   eShop
      mandatory
         Security
            alternative
               High {Price 100}
               Standard {Price 50}
         Catalogue cardinality [1..5] {Price 30}
   ...
```

4.4. Type level

Listing 5 shows the last version of our `eShop` with all language levels included. Here, we newly added the *Type*-level, which introduces features with the following types: integer, float, and string. Note that any feature can still be deselected even if it is not Boolean. For, instance the customer can now configure an integer feature `Items in Basket`, which can be used to limit the maximum number of items a customer can put in his basket at the same time. A cross-tree constraint ensures that the maximum number of items is higher than zero. Further, `Browser` is now a string feature where the URL can be directly configured. Another cross-tree constraint denotes that the URL may not be longer than 30 characters. The used *len*-function is part of the *string-constraints* minor level which also introduces equality checks between strings. Note that we also replaced the two lines for specifying both minor levels of the *Arithmetic* level with a wildcard `Arithmetic.*`.

4.5. Import mechanism

With thousands of features and constraints in practice (Lotufo et al., 2010; Kübler et al., 2010; Sundermann et al., 2023a), FMs are often hard to overview. Further, stakeholders often only need to consider a subset of the FM. To simplify managing large FMs and focusing on parts of interest, UVL provides a mechanism for decomposing models into subparts that can then be imported in an overall model if needed.

Listing 6 showcases the *import* mechanism of UVL where we have the `Platform` subtree (Listing 7) and the `Security` subtree (Listing 8 as separate files. Those subtrees are imported using the *imports*-keyword. Imports are specified using a relative file path to the imported UVL model. For instance, `platform` refers to a file in the same directory named `platform.uvl`. Non-trivial paths can be specified with a Python-like dot notation (e.g., submodels.platform). Imports can also be given an alias with the *as* keyword. The submodel can then be attached to an arbitrary location in the feature tree by referencing its root feature (e.g., `pl.Platform`). In the cross-tree constraints all features of submodels can be referenced using the submodels' namespace. The shown model (Listing 6) is equivalent to Listing 5. Semantically, the feature reference in the composed model is expanded to include the entire subtree. For instance, `pl.Platform` references the entire FM in Listing 7. Also, all cross-tree constraints in the imported submodels are applied for the composed model. Cross-tree constraints in the composed can reference features from imported submodels using the filename or alias and the feature name. For example, in line 21 `pl."Mobile App"` is referenced. The import mechanism may have the following two advantages for our running example. First, Listing 6 is shorter and easier to overview than the entire model shown in Listing 5. Second, a developer only responsible for platform or security development can separately work on the submodels Listing 7 and Listing 8, respectively.

Listing 5: UVL Running Example: Typed Features

```
1  include
2     Boolean.group-cardinality
3     Arithmetic.*
4     Type.string-constraints
5
6  features
7     eShop
8        mandatory
9           Security
10             alternative
11                High {Price 100}
12                Standard {Price 50}
13          Catalogue cardinality [1..5] {Price 30}
14          Integer "Items in Basket"
15       optional
16          SEO {Price 40}
17          Payment
18             alternative
19                "Bank Transfer" {Price 10, SEPA true}
20                "Credit Card" {Price 20}
21          Platform
22             [2..3]
23                "Desktop App" {Price 70}
24                Boolean "Mobile App" {Price 80}
25                String Browser {Price 20}
26
27 constraints
28    !("Bank Transfer" & "Mobile App")
29    "Credit Card" => High
30    sum(Price) < 200
31    0 < "Items in Basket"
32    len(Browser) < 30
```

Listing 6: UVL Running Example: Import Mechanism

```
1  imports
2     platform as pl
3     security
4
5  features
6     eShop
7        mandatory
8           security.Security
9           Catalogue cardinality [1..5] {Price 30}
10          Integer "Items in Basket"
11       optional
12          SEO {Price 40}
13          Payment
14             alternative
15                "Bank Transfer" {Price 10, SEPA true}
16                "Credit Card" {Price 20}
17          pl.Platform
18
19
20 constraints
21    !("Bank Transfer" & pl."Mobile App")
22    "Credit Card" => security.High
23    sum(Price) < 200
24    "Items in Basket" > 0
```

Summary. UVL provides a simple core language and an import mechanism that enables decomposing models into manageable small submodels to tackle its design goal *simplicity*. Additional language levels provide more *expressiveness* with constructs to specify cardinalities, different constraints over numeric values, typed features, and constraints over strings. The design of the language levels is *extensible* to allow

Listing 7: UVL Running Example: Platform Submodel

```
features
    Platform
        [2..3]
            "Desktop App" {Price 70}
            Boolean "Mobile App" {Price 80}
            String Browser {Price 20}

constraints
    len(Browser) < 30
```

Listing 8: UVL Running Example: Security Submodel

```
features
    Security
        alternative
            High {Price 100}
            Standard {Price 50}
```

different users to tailor their UVL models to their use case and tool limitations. We used this section to introduce UVL with an example, in Section 5 we define UVL models more formally and discuss the semantics of different constraints.

5. Syntax & semantics: Language specification

In this section, we discuss the syntax and semantics of UVL more formally. The goal is to clarify possible ambiguities and provide clear guidelines on how to interpret UVL and work with the language. Note that we use some concepts here requiring computer science background to understand.

5.1. UVL syntax

Fig. 4 shows a simplified view on the abstract syntax of a UVL model in form of a meta-model (more details on concrete parts of the meta-model will be elaborated later). A UVL model consists of four major parts: *imports*, *language levels*, *feature tree*, and *cross-tree constraints*. In the following, we explain the four major parts in more detail and the language constructs that can be used within.

Imports. As discussed in Section 3, decomposing a feature model into smaller sub-parts is beneficial. Still, knowledge about cross-dependencies between those sub-parts needs to be maintained as they may impact the configuration space. With UVL, we support composition of various smaller sub-models with an import mechanism. Hereby, another UVL model can be imported via `import submodel`. Then, the submodel can be referenced at an arbitrary location in the feature tree with `submodel.Root`. Note that `Root` is the name of the root feature here. While the composed model only contains one line for adding the root feature, this is semantically equivalent of copying the entire sub-model at this location. Cross-tree constraints of the sub-model also apply for the composed model. Constraints between features of different sub-models can be specified using the same syntax as in the feature tree (e.g., `submodel1.A & submodel2.B`). For each import, an alias can be specified with the *as*-keyword (e.g., `import submodel1 as s1`). Features can then be referenced with `s1.A`. Submodels in other, possibly nested, directories can be referenced with `<dir1>.<dir2>.<uvlfile>`.

Language levels. Language levels can be explicitly specified with the *include* keyword. The included language levels are listed in separate lines using the syntax `<major>.(<minor>|*)?`. So, one line can either specify a major level (`<major>`), a minor level (`<major>.<minor>`, or all minor levels (`<major>.*`). If a developer violates the language levels by adding an unsupported construct, the parser would provide a warning or error to him. Note that using a minor level always includes the respective major level. By default, all language levels are included. Hence, not specifying any level includes enables the full expressiveness of UVL.

Fig. 3 shows the language levels currently supported in UVL and the language constructs they include. The three major levels *Boolean*, *Arithmetic*, *Type* encapsulate language constructs that can be reasoned about with a specific reasoning engine. For instance, UVL models of the Boolean level should be straightforward to encode as a SAT instance (e.g., CNF) (Batory, 2005; Kuiter et al., 2022). In contrast, the Arithmetic level can be directly represented as SMT (De Moura and Bjørner, 2011) or CP (Jussien et al., 2008) problem, but requires further processing to be encoded as SAT instance.

Feature tree. The UVL feature tree consists of two main elements: *features* and *groups*. The tree requires exactly one root feature. Each feature may have an arbitrary number of groups which in turn may have an arbitrary number of features each. The relationship between features and groups are denoted with indentation. For a feature, its corresponding groups are indented by one line and vice versa for groups. A feature always requires a unique *name* as identifier. Here, the identifier needs to be enclosed by quotation marks if the used symbols may introduce an ambiguity in the UVL model.[6] Each feature can have a *feature cardinality* [n..m], which denotes that the feature can be selected between n and m times. Also, a list of *attributes* {att1, att2, ...} can be attached. There are five feature group types supported in UVL as seen in Fig. 5. *Optional*, *mandatory*, *or*, and *alternative* are part of the core (Boolean) level, while *group cardinality* is a Boolean minor level.

Feature attributes. For each feature, an arbitrary number of attributes can be attached. Generally, attributes are key–value pairs with the key being an identifier and the value being of one of the types shown in Fig. 6. One exception is that it is allowed to only specify a key, which is then considered as Boolean attribute with true as value. The attributes of a feature are declared in curly brackets as follows: `{<key1> <val1>, <key2> <val2>}`. Nested *attribute lists* can be specified with `{<key> {<key1> <val1> ...}}`. Types of attributes are not explicitly stated but rather inferred from the value. String constants (i.e., values of string attributes) are specified with single quotation marks to prevent ambiguities with feature names.

Constraints. The constraints part of a UVL model consists of a list of constraints that can evaluate to either true or false (see Fig. 7). A valid configuration needs to satisfy every attached constraint. In UVL, constraints are mostly based on common propositional logic operators, namely `!` (not), `&` (and), `|` (or), `=>` (implies), `<=>` (equals), and brackets. These operators combine Boolean variables, Boolean constants (i.e., true or false), or predicates(cf. Fig. 8), which each need to evaluate to a Boolean. The Boolean variables can refer to either a feature name or a Boolean-attribute key.

Predicates. Predicates are used in UVL to specify dependencies based on string and numerical values. As shown in Fig. 8, each predicate has exactly one operator of equals (== in UVL), greater (>), lesser (<), unequal (!=), greater equal (>=), and lesser equal (<=). Note that each of these operators evaluates to a Boolean value. Those operators connect two terms, which are illustrated in Fig. 9.

[6] Identifiers not matching `[a-zA-Z0-9_]*[a-zA-Z_][a-zA-Z0-9_]*` must be protected.

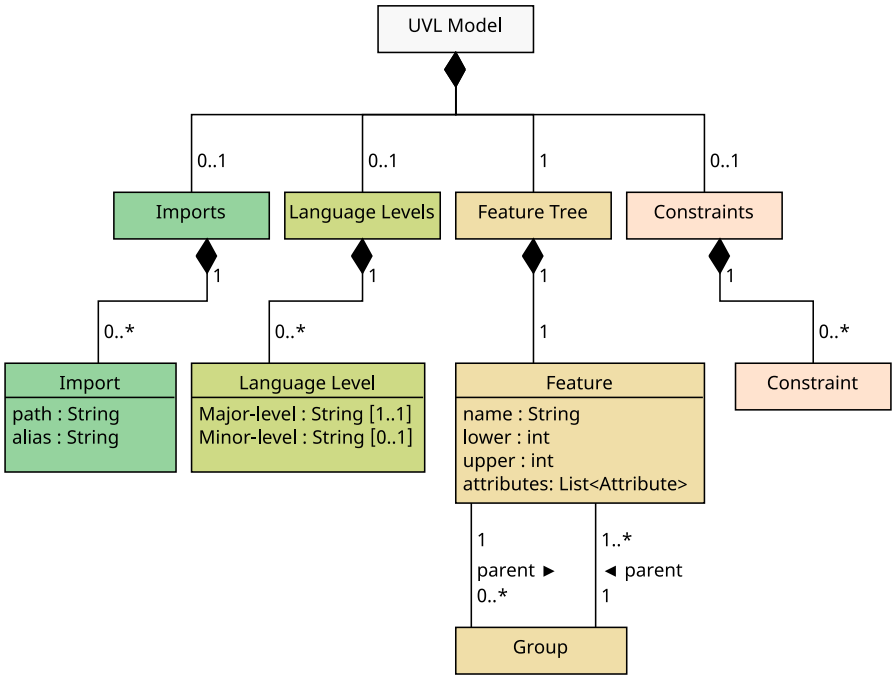

Fig. 4. UVL meta model.

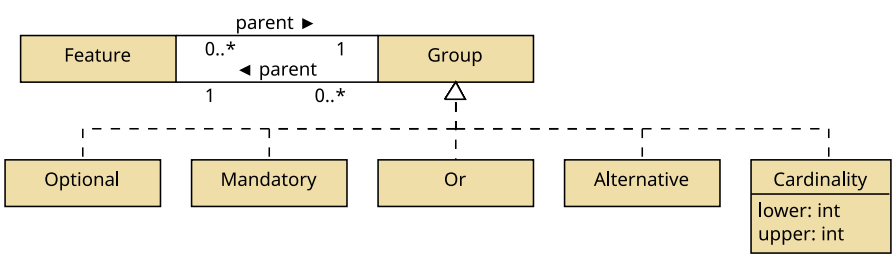

Fig. 5. UVL feature-group types.

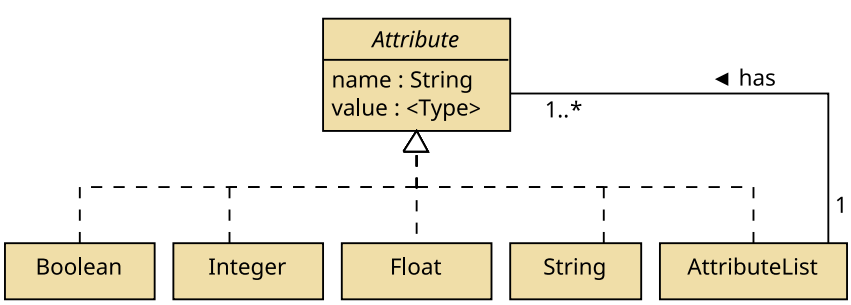

Fig. 6. UVL feature attributes.

Terms can only be used within predicates in UVL. A term can be (1) a reference to a variable, (2) a constant, (3) a function, or (4) a binary expression. The referenced variables can be either features or attributes. Constants can be either strings or numeric values. String constants are always enclosed with single quotation marks to prevent ambiguities with references. Otherwise, it would be impossible to distinguish between a string constant matching a feature name and said feature. For functions, *sum*, *average* are currently supported for numeric values and *length* for strings. The binary expression can connect two numeric values with simple arithmetic operators, namely add (+ in UVL), subtract (−), multiply (∗), and division (/).

5.2. Constraint semantics

In this section, we discuss the semantics of constraints in UVL considering on how they affect the set of valid configurations. Our goal here is to clarify potential ambiguities in the semantics of specific constraints. Table 1 shows a formal definition of the restrictions different constraints impose on the *configuration space* VC (i.e., the set of valid, complete configurations modelled by a UVL model). For $C = (I, E)$, I is the set of included features and E of excluded features. Further, f is a feature, $p(f)$ the parent feature of f, $s(f, C)$ the selection status of f in C, $\text{card}(f, C)$ the cardinality of f in C. The selection status is a function s : (feature, configuration) $\rightarrow \{0, 1\}$ that maps a feature selected (1) or deselected (0). The cardinality of a feature $\text{card}(f, C)$

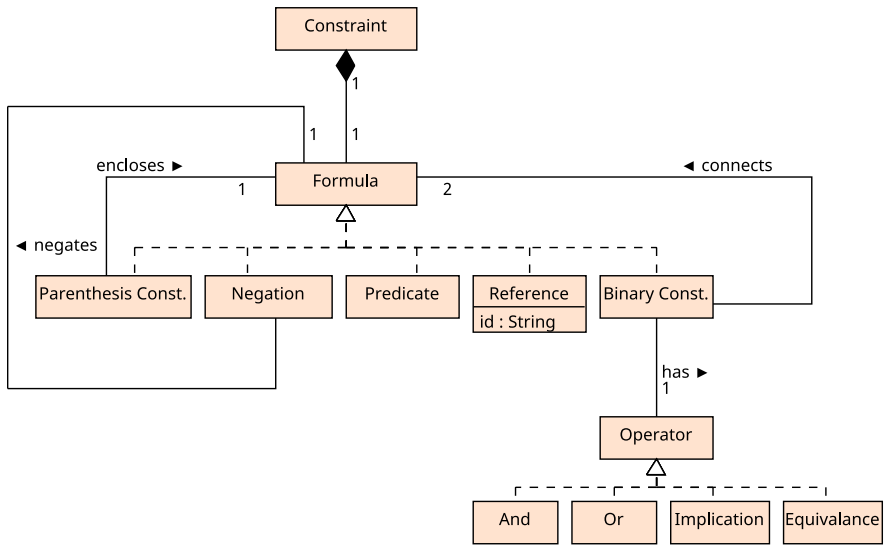

Fig. 7. UVL cross-tree constraint.

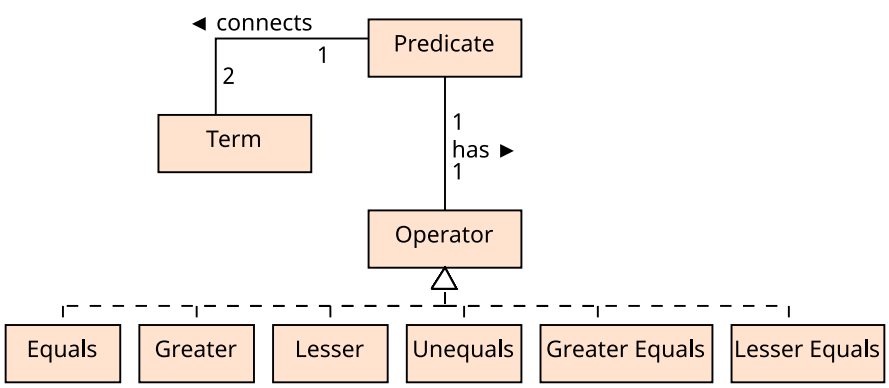

Fig. 8. UVL predicates.

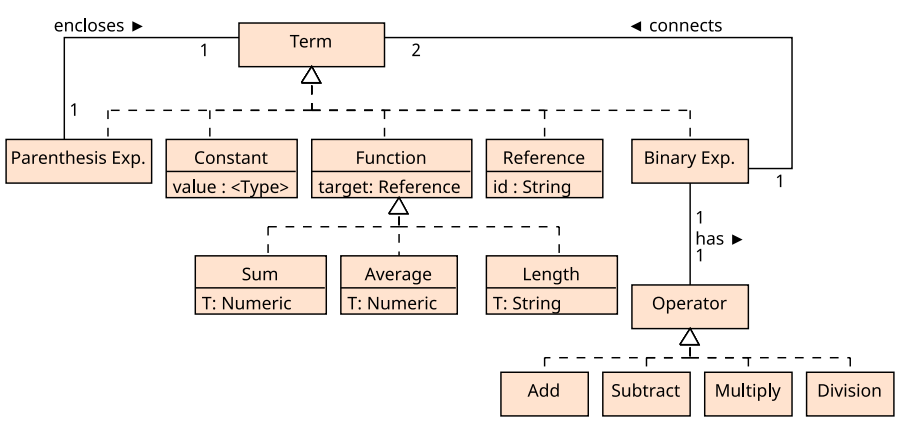

Fig. 9. UVL terms.

describes the selection of a feature as integer number. Note that features without denoted cardinality can only have the values 0 and 1. G is a set of features and ϕ is an arbitrary logical formula. The semantics of the different constraints are equivalent to the descriptions in Section 2.

Feature cardinality. The semantics of feature cardinality are not straightforward and drive discussions in research (Czarnecki and Kim, 2005). Generally in UVL, we consider subtrees induced by a cardinality as clones that can be configured separately. However, the current syntax does not support referencing specific clones, which requires a default behaviour in handling feature cardinalities in other constraints. In the following, we discuss how we interpret interactions between feature cardinality and cross-tree constraints. We use Listing 9 to illustrate the interactions of feature cardinalities within a simple UVL model. The feature A can be selected between two and three times. As a consequence, the entire subtree including B and C can be configured up-to three times. However, it is not straightforward to interpret both cross-tree constraints. If we select A two times and also select D, do we need to select C for every subtree clone as consequence of the

Table 1
Constraint semantics.

Constraint	If $C = (I, E) \in VC$ this needs to hold
Feature f	$s(p(f), C) \geq s(f, C)$
Root f	$s(f, C) = 1$
Mandatory f	$s(p(f), C) = s(f, C)$
Group Cardinality [n..m] G	$n \leq \sum_{f \in G} s(f, C) \leq m$
Alternative G	$\sum_{f \in G} s(f, C) = 1$
Or G	$\sum_{f \in G} s(f, C) \geq 1$
Feature Cardinality [n..m] f	$n \leq \text{card}(f, C) \leq m$
Cross-tree constraint ϕ	$\text{SAT}(\phi \wedge \bigwedge_{i \in I} i \wedge \bigwedge_{e \in E} \neg e)$

Listing 9: Cardinality Interactions in UVL

```
features
   R
      optional
         A cardinality [2..3]
            optional
               B
               C
         D

constraints
   B => C
   !D | (C & B)
```

implication $D \Rightarrow C$? Or do we need to select at least one C? In the following, we explain on how we interpret feature cardinalities with UVL.

Listing 10 shows a UVL model where we resolved the feature cardinality to illustrate its semantics. The feature cardinality consists of three clones of the original subtree within a group cardinality that ensures that [2..3] of those subtrees must be selected. Listing 10 also depicts the default interpretation of cross-tree constraints with features that are cloned in UVL. First, we create *contextual clone constraints* to ensure the configurability of the clones behaves as expected. Here, we have copies of the cross-tree constraints containing a feature from the feature cardinality subtree for each clone (i.e., A_1–A_3). Each of those constraints is only applied in its respective context (i.e., its subtree is selected). For instance, (B_2 => C_2) only needs to be satisfied when the second instance of A is selected. This is realised in UVL with the implication A_2 => (B_2 => C_2). If the constraint would always be applied (i.e., only having right side of implication), the constraints would automatically apply for every clone. In particular, when D is selected, it would be required to select every C_i and in consequence every A_i. Hence, selecting D would enforce selecting three instances of A. In addition to the contextual clone constraints, we have one copy of constraints containing features that are part of the cardinality subtree and ones that are not. Here, we replace each occurrence of a cardinality feature f with an or over each of the clone features $f_1 \vee \ldots \vee f_m$. For an example, see the first cross-tree constraint in Listing 10. The idea of this constraint is to ensure constraints with other features are met with at least for one clone. We assume that this often matches the expectation for constraints such as D => C, where one would expect that selecting D requires to have a C.

5.3. Conversion strategies

With the extensible language design of UVL, another problem arises: the *exchange* of UVL models between tools that employ different language levels. If tool A supports a higher language level than tool B, feature models of tool A cannot be used in tool B. This may even be

Listing 10: Cardinality Semantics Different Versions

```
features
   A
      [2..3]
         A_1
            optional
               B_1
               C_1
         A_2
            optional
               B_2
               C_2
         A_3
            optional
               B_3
               C_3
   D

constraints
   A_1 => (B_1 => C_1)
   A_2 => (B_2 => C_2)
   A_3 => (B_3 => C_3)
   A_1 => (!D | (C_1 & B_1))
   A_2 => (!D | (C_2 & B_2))
   A_3 => (!D | (C_3 & B_3))
   !D | ((C_1 | C_2 | C_3) & (B_1 | B_2 | B_3))
```

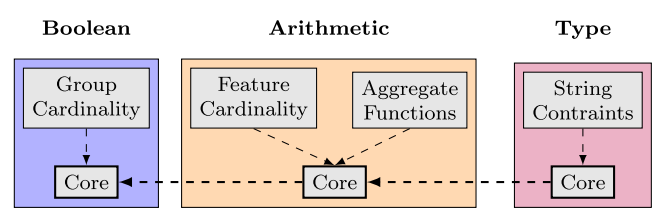

Fig. 10. Language levels in UVL.

an issue for a single developer, as different variability modelling tools have different capabilities and advantages (Meinicke et al., 2017; Acher et al., 2013; Mendonça et al., 2009a; Loth et al., 2023).

With UVL, we tackle the issue of *exchanging* models with different language levels by using *conversion strategies*. Note that we only consider translation between different levels of UVL here in contrast to conversions to other variability languages as performed by tools such as TRAVART (Feichtinger et al., 2021). Fig. 10 shows the current language level hierarchy of UVL and conversion strategies between them. A conversion takes a UVL model of a certain level and converts it to a UVL model of the next lower level by replacing the constructs with semantically equivalent constructs from the lower level. The possible conversions and their directions are marked in Fig. 10 with dashed arrows. For a minor level, its next lower level is its major level (e.g., Boolean for group cardinality). For a major level, its next lower level is the most expressive major language level that is *included* (cf. Fig. 3) by the level to translate. In Fig. 10 the hierarchy is illustrated from right to left with the rightmost (Type) having the highest hierarchy. Since we have a conversion to the next lower level for every level, we can transitively convert higher language constructs to lower ones. Also, we only need to implement one additional conversion when introducing a new language level. Note that models may exponentially grow in size when converted.

Table 2 shows the conversion strategies applied in UVL. The rows show the source level, the target level when converting the source level, an illustration of the construct in the source level, and the result of the respective conversion. For *Group Cardinality* and *Arithmetic*-level, we provide a specific example in the table for comprehensibility of

Table 2
Conversion Strategies.

Source	Target	Original	Converted
Boolean.group-cardinality	Boolean	[2..3] f_1 f_2 f_3	$(f_1 \wedge f_2 \wedge \neg f_3) \vee (f_1 \wedge \neg f_2 \wedge f_3)$ $\vee(\neg f_1 \wedge f_2 \wedge f_3)$
Arithmetic	Boolean	f_1.a + f_2.b + f_3.c < 20 f_1.a = 5, f_2.b = 10, f_2.b = 16	$\neg f_1 \vee \neg(f_2 \wedge f_3)$
Arithmetic.feature-cardinality	Arithmetic	a cardinality [n..m]	a [n..m] a_1 ... a_m
Arithmetic.aggregate-function	Arithmetic	sum(a) avg(a)	$f_1.a + f_2.a + ... + f_n.a$ $\frac{(f_1.a+f_2.a+...+f_n.a)}{f_1+f_2+...+f_n}$
Type	Arithmetic	Integer f Float f String f Boolean f	f {Integer 0} f {Float 0} f {String 0} f
Type.string-constraints	Type	f == "Fun" len(f)	Drop Drop

the conversion strategy. For conversions of both concepts, we specify the valid partial configurations over the features involved according to the respective constraint. Involved are the features in the Group Cardinality and the features occurring in the predicate, respectively. For *Feature Cardinality*, we expand the Feature Cardinality by introducing the respective number of clone (cf. Section 5.2). Note that we directly convert the Group Cardinality according to the presented conversion strategy, since conversions do not rely on other minor levels. For *Aggregate Functions*, we expand the function by applying the operation on all features that have the respective attribute. For the *Type*-level, we transform the features with different types to a boolean feature which has an attribute according to the type of the original feature. Currently, we just drop the string constraints from the *Type*-level, since we found no way to represent those constraints with constructs from other levels.

6. UVL implementation and integration with other tools

In this section, we introduce the reference parser implementation of the UVL language. Further, we outline other tools integrating the UVL for various purposes, including graphical editing, textual editing, analysis, configuration, and transformation.

6.1. UVL parser

The current UVL parser (Sundermann et al., 2023b) extends the previous implementations by the presented language levels discussed in the previous sections. The parser is based on ANTLR (Parr, 2013) and is available as an open-source implementation in Java and Python.[7] Listing 11 shows the syntax of a UVL model implemented in the parser as a simplified grammar in an EBNF-like notation. The parser implements the necessary conversion strategies between *Boolean*, *Arithmetic*, and *Type*-level of UVL (cf. Table 2). A conversion only works from a higher level to the next lower level to allow importing more expressive UVL models in tools that only build on lower levels. Thus, concepts of the *Type*-level are converted to the *Arithmetic*-level and these concepts are then converted to the core *Boolean*-level. Fig. 10 shows the implemented language concepts, organised per level and highlighting the conversion strategies. In previous work, we implemented parsers based on Clojure (Sundermann et al., 2021a) and Python (Galindo and Benavides, 2020), but both those parsers are limited to the Boolean level of UVL.

[7] UVL Parser – https://github.com/Universal-Variability-Language/uvl-parser

6.2. Available tooling

Several tools, such as FeatureIDE (Meinicke et al., 2017; Sundermann et al., 2021b), flamapy (Galindo et al., 2023), TRAVART (Feichtinger et al., 2021), or variability.dev (Heß et al., 2024) have integrated UVL for different purposes. Most tools either enable graphical (Heß et al., 2024; Meinicke et al., 2017; Sundermann et al., 2021b) or textual (Loth et al., 2023) editing, analysis (Galindo et al., 2023; Heß et al., 2024; Loth et al., 2023; Meinicke et al., 2017; Munoz et al., 2023; Sundermann et al., 2021b), transformation (Feichtinger et al., 2021; Munoz et al., 2023), or configuration (Galindo et al., 2023; Heß et al., 2024; Loth et al., 2023; Meinicke et al., 2017; Sundermann et al., 2021b) of UVL models. Some tools, such as FeatureIDE (Meinicke et al., 2017; Sundermann et al., 2021b), flamapy (Galindo et al., 2023) or Nemo (Munoz et al., 2023) do support multiple purposes at once. For instance, the UVLS (Loth et al., 2023) provides an implementation of the language server protocol to enable easy integration into existing tools and additionally the configuration of UVL models. Other tools use UVL to facilitate variability model interoperability via the transformation of variability artefacts into UVL (Feichtinger et al., 2021; Munoz et al., 2023). Further, some tools, such as pure::variants (pure::systems, 2017; Romano et al., 2022), ddueruem (Heß et al., 2022b), FM Fact Label (Horcas et al., 2022a), or V4rdiac (Fadhlillah et al., 2022), integrated UVL or one of the tools supporting UVL to expand the range of supported variability artefacts in their respective tool. Not all tools support all language levels of UVL. In the following, we discuss the respective tools, their focus, and which UVL language level they support. Table 3 summarises the discussed tools.

6.2.1. Graphical editing

FeatureIDE (Meinicke et al., 2017; Sundermann et al., 2021b) is the de-facto standard for graphical editing of feature models. The Eclipse-based tool allows defining feature models using the core of the UVL *Boolean*-level. Thus, UVL models created in FeatureIDE may consist of optional and mandatory features, and each feature may consist of a set of optional and mandatory features themselves, or a single alternative, or an or group. However, FeatureIDE also does not support feature and group cardinalities. Constraints are limited to propositional logic constraints. Feature models created with FeatureIDE are usually serialised using an XML format. However, the serialisation can be changed to the UVL format (Sundermann et al., 2021b), using the UVLFeatureModelFormat class.

Listing 11: Simplified UVL Grammar in EBNF Notation

```
featureModel: includes? NEWLINE? imports? NEWLINE? features? NEWLINE? constraints? EOF;

includes: 'include' NEWLINE INDENT includeLine* DEDENT;
includeLine: languageLevel NEWLINE;

imports: 'imports' NEWLINE INDENT importLine* DEDENT;
importLine: ns=reference ('as' alias=reference)? NEWLINE;

features: 'features' NEWLINE INDENT feature DEDENT;

group
    : ORGROUP groupSpec           # OrGroup
    | ALTERNATIVE groupSpec # AlternativeGroup
    | OPTIONAL groupSpec    # OptionalGroup
    | MANDATORY groupSpec   # MandatoryGroup
    | CARDINALITY groupSpec    # CardinalityGroup
    ;

groupSpec: NEWLINE INDENT feature+ DEDENT;

feature: featureType? reference featureCardinality? attributes? NEWLINE (INDENT group+ DEDENT)?;

featureCardinality: 'cardinality' CARDINALITY;

attributes: OPEN_BRACE (attribute (COMMA attribute)*)? CLOSE_BRACE;

attribute
    : valueAttribute
    | constraintAttribute;

valueAttribute: key value?;

key: id;
value: BOOLEAN | FLOAT | INTEGER | STRING | attributes | vector;
vector: OPEN_BRACK (value (COMMA value)*)? CLOSE_BRACK;

constraintAttribute
    : 'constraint' constraint               # SingleConstraintAttribute
    | 'constraints' constraintList          # ListConstraintAttribute
    ;
constraintList: OPEN_BRACK (constraint (COMMA constraint)*)? CLOSE_BRACK;

constraints: 'constraints' NEWLINE INDENT constraintLine* DEDENT;

constraintLine: constraint NEWLINE;

constraint
    : equation                              # EquationConstraint
    | reference                             # LiteralConstraint
    | OPEN_PAREN constraint CLOSE_PAREN     # ParenthesisConstraint
    | NOT constraint                        # NotConstraint
    | constraint AND constraint             # AndConstraint
    | constraint OR constraint              # OrConstraint
    | constraint IMPLICATION constraint     # ImplicationConstraint
    | constraint EQUIVALENCE constraint     # EquivalenceConstraint
        ;

equation
    : expression EQUAL expression           # EqualEquation
    | expression LOWER expression           # LowerEquation
    | expression GREATER expression         # GreaterEquation
    | expression LOWER_EQUALS expression    # LowerEqualsEquation
    | expression GREATER_EQUALS expression  # GreaterEqualsEquation
    | expression NOT_EQUALS expression      # NotEqualsEquation
    ;

expression:
    FLOAT                                   # FloatLiteralExpression
    | INTEGER                               # IntegerLiteralExpression
    | STRING                                # StringLiteralExpression
    | aggregateFunction                     # AggregateFunctionExpression
    | reference                             # LiteralExpression
    | OPEN_PAREN expression CLOSE_PAREN     # BracketExpression
    | expression ADD expression             # AddExpression
    | expression SUB expression             # SubExpression
    | expression MUL expression             # MulExpression
    | expression DIV expression             # DivExpression
    ;
```

Table 3
Tools integrating UVL either directly (upper part) or indirectly (lower part).

Tool	Graphical Editing	Textual Editing	Configuration	Analysis	Transformation	Supported Lang. Levels
FeatureIDE (Meinicke et al., 2017)	✓	✗	✓	✓	✗	*Boolean*
flamapy (Galindo et al., 2023)	✗	✗	✓	✓	✓	*Boolean*
Nemo (Munoz et al., 2023)	✗	✗	✗	✓	✓	*Boolean*
TRAVART (Feichtinger et al., 2021)	✗	✗	✗	✗	✓	*All*
UVLS (Loth et al., 2023)	✗	✓	✓	✓	✗	*All*
variability.dev (Heß et al., 2024)	✓	✗	✓	✓	✗	*Boolean*
ddueruem (Heß et al., 2022b)	✗	✗	✗	✓	✗	*Boolean*
FM Fact Level (Horcas et al., 2022a)	✗	✗	✗	✓	✗	*Boolean*
pure::variants (pure::systems, 2017)	✓	✗	✓	✓	✗	*Boolean*
V4rdiac (Fadhlillah et al., 2022)	✗	✗	✓	✗	✗	*All*

The web-based feature-modelling tool variability.dev (Heß et al., 2024)[8] builds on the ddueruem (Heß et al., 2022b) analysis wrapper and the FeatureIDE (Meinicke et al., 2017) library. Thus, the expressiveness of the created UVL models is the same as those created with FeatureIDE and limited to the core of the UVL *Boolean*-level. However, using variability.dev, users have a low entry point for experimenting with feature modelling as users do not have to install a complete Eclipse-based application. Additionally, variability.dev allows collaborative editing of feature models. Created feature models can be downloaded either as a graphical image (SVG) or as a FeatureIDE XML file.

6.2.2. *Textual editing*

For textual editing of UVL models, Loth et al. (2023) implemented the language server protocol for UVL in the tool UVLS. The language server protocol enables important language features, such as syntax highlighting, via a standardised interface. Thus, it can be integrated

[8] variability.dev – https://variability.dev/

into common development environments, e.g., Visual Studio Code.[9] The current implementation of the language server protocol supports all language levels of UVL and features several analysis techniques (Loth et al., 2023) to enhance the textual editing of UVL models (cf. Section 6.2.3). For instance, UVLS checks whether the created UVL model is syntactically and semantically correct, e.g., avoiding void feature models. Furthermore, UVLS allows the configuration of a UVL model in a simplified configuration editor, similar to the one provided by FeatureIDE (Meinicke et al., 2017; Sundermann et al., 2021b).

6.2.3. Analysis

FeatureIDE (Meinicke et al., 2017; Sundermann et al., 2021b) cannot only be used for graphical editing of feature models, but also for analysing them. Hence, FeatureIDE also enables the analysis of UVL models. However, the analysis is limited to the core of the *Boolean*-level, as this is the level supported by FeatureIDE (cf. Section 6.2.1).

The language server protocol implementation UVLS (Loth et al., 2023) provides syntactical and semantical analysis capabilities. For the syntactic check of a given UVL feature model, UVLS utilises the tree-sitter parser generator tool.[10] UVLS then checks if the tree-sitter parser accepts the given UVL model as a valid input. For the semantic analysis of a given UVL feature model, UVLS utilises the Z3 solver (de Moura and Bjørner, 2008). The SMT solver allows detecting if a UVL model does not allow valid configurations or contains any dead features or contradicting or redundant constraints.

flamapy (Galindo et al., 2023) is a Python-based analysis framework for feature models.[11] The tool is plugin-based, utilising a core plugin orchestrating the execution of other plugins and also providing the hooks and frozen points of the framework (Matos and Fernandes, 2006). Besides the core plugin, flamapy provides a feature model plugin, which supports the core of UVLs' *Boolean*-level and provides translations for PySAT, BDD support, and various input formats such as FeatureIDE (Meinicke et al., 2017) and S.P.L.O.T. (Mendonça et al., 2009a). Currently, flamapy supports multiple different solvers via the support of the PySAT4 metasolver,[12] BDDs (Heradio et al., 2022) and dependency graphs (Márquez et al., 2022).

Nemo (Munoz et al., 2023) allows counting valid configurations of numerical feature models via bit blasting (Munoz et al., 2019). The tool currently supports UVL as an input and output format (cf. Section 6.2.4). Therefore, Nemo utilises the *Boolean*-level of UVL, including group cardinalities. Based on the bit-blasted UVL model #SAT solvers and BDD solvers (Heradio et al., 2022) are executed to count the number of valid configurations of the resulting model.

The online tool variability.dev (Heß et al., 2024) uses the analysis wrapper ddueruem (Heß et al., 2022b) and the FeatureIDE (Meinicke et al., 2017) library to perform basic analysis on a created feature model. For instance, variability.dev detects dead features in a feature tree, or faulty configurations in the configuration editor (cf. Section 6.2.5).

6.2.4. Transformation

TRAVART (Feichtinger et al., 2021) is a plugin-based variability model transformation environment.[13] At its core, TRAVART uses UVL as the pivot model. As the tool builds on the current Java implementation, TRAVART supports all language levels of UVL. Each plugin implements transformations between one variability artefact type and the UVL. These transformations are usually built by mapping core concepts of the supported variability model type onto the core concepts of UVL and vice versa (Feichtinger et al., 2022; Feichtinger and Rabiser, 2020a,b). For instance, in the available plugin for the DOPLER (Dhungana et al., 2011) decision modelling approach, a decision is mapped to a feature in the UVL. Also, a rule in the DOPLER decision model is mapped to either a feature property (mandatory), the feature model tree, or a constraint (Feichtinger and Rabiser, 2020b). In the opposite direction, the hierarchy of the UVL feature model tree is captured via the visibility conditions of the DOPLER decision model.

Nemo (Munoz et al., 2023) translates numerical feature models into UVL feature models using bit blasting (Munoz et al., 2019). Therefore, the bit-blasted numerical feature model is captured either as a DIMACS file, from which a UVL model is created or directly as a UVL model. Using Nemo the created UVL model can then be analysed (cf. Section 6.2.3).

6.2.5. Configuration

FeatureIDE (Meinicke et al., 2017; Sundermann et al., 2021b) also supports the configuration of feature models. Hence, FeatureIDE also enables the configuration of UVL models which support the core of the *Boolean*-level.

flamapy (Galindo et al., 2023)[14] utilises the configuration of UVL models, which support the core of the *Boolean*-level. flamapy uses the capability to validate if a configuration is valid for the given UVL model or to count valid configurations via state-of-the-art SAT solvers.

UVLS (Loth et al., 2023) supports configuring a given UVL model using a dedicated editor. The configuration editor supports the configuration of UVL models of all language levels. Therefore, UVLS presents a decision for each feature and its feature attributes to configure a configuration. The editor indicates if the given values for these features and their attributes still provide a basis for a valid configuration for the UVL model.

The online tool variability.dev (Heß et al., 2024) allows configuring created feature models using its configuration editor. The configuration editor supports the configuration of UVL models using the core of the *Boolean*-level. By default, the editor ensures that the selected configuration is valid, but also allows the configuration of invalid configurations. Configurations can be downloaded as a FeatureIDE configuration.

6.2.6. Others

UVL has also been either directly or indirectly, i.e., via one of the tools mentioned above, integrated into other tools. For instance, pure::variants (pure::systems, 2017) or ddueruem (Heß et al., 2022b) support UVL via import and export capabilities (Romano et al., 2022). Similarly, FM Fact Label (Horcas et al., 2022a) facilitates the visualisation of feature model metrics and supports common feature model formats, such as FeatureIDE (Meinicke et al., 2017), S.P.L.O.T. Mendonça et al. (2009a) or UVL. Other tools, such as V4rdiac (Fadhlillah et al., 2022), integrated TRAVART (Feichtinger et al., 2021) to achieve variability model interoperability via the UVL or to facilitate the configuration of Cyber-Physical Production Systems (Meixner et al., 2024). The UVLGenerator can be used to generate UVL models whose structural properties can be customised according to the user's requirements (Sundermann et al., 2024). Last but not least, UVLHub an open repository with UVL datasets is available (Romero-Organvidez et al., 2024b).[15]

7. Discussion, open challenges and future work

To increase the adoption of UVL, its acceptance in industry is essential. To achieve that, we need to address the challenges that industry is having regarding variability modelling.

In 2020, Berger et al. provided some updates on industry challenges in SPLE (Berger et al., 2020) elicited earlier. At the SPLC 2023 Industry

[9] UVLS: https://marketplace.visualstudio.com/items?itemName=caradhras.uvls-code
[10] tree-sitter: https://tree-sitter.github.io/tree-sitter/
[11] flamapy: https://flamapy.github.io/
[12] PySAT: https://pysathq.github.io/
[13] TraVarT: https://github.com/SECPS/TraVarT
[14] https://www.flamapy.org/
[15] https://www.uvlhub.io/

Challenges Workshop (Becker et al., 2024), 9 companies presented their challenges regarding variability management and systems and software product lines and discussed research opportunities. Addressing the challenges elicited in these recent works is essential for UVL to ensure adoption by industry. Of the many challenges discussed, especially the need to support multi product lines and system of systems product lines, efficient PL verification and validation, and tool support for integrated variability management across disciplines are relevant for the further development of UVL. The already available tooling and the extensibility of UVL should already help to address these challenges, however, further work needs to be done.

A recent paper (Rabiser, 2024) described specific challenges for UVL industry adoption, which we include:

- work with industry to empirically validate UVL and demonstrate it actually works for realistic cases. Extend or adapt UVL if necessary. Create demonstrators.
- develop extended tool support for modelling and configuration including generators for domain-specific artefacts.
- bridge the gap between UVL models and web-based (sales) configurators (see initial work by Abbasi et al. (2013)).
- develop flexible mapping concept to support mapping of UVL features to solution space artefacts.
- develop consistency checking support intra- and inter-UVL models as well as between UVL models and artefacts.
- support the verification of UVL models and configurations
- support the automated creation of UVL models based on analysing existing variability information and existing artefacts (Kästner et al., 2013) to extract variability.
- integrate UVL with tools used in industry such as ALM/PLM tools.
- support product line maintenance and evolution, e.g., develop automated refactoring support and a proper versioning concept and integrations with version management frameworks.
- work on the scalability of UVL to real-world systems. The multi-modelling concept of UVL thus should only be seen as a first step in this direction.
- investigate different visualisations of UVL models and configurations.
- provide material to train users as well as the advertise UVL.
- define UVL design patterns and guidelines (Nesić et al., 2019).

Some of the challenges are being addressed and there are some solutions available. In future work, we plan to work on some of these challenges and also discuss them further with industry based on first case studies and demonstrators. We envision that this paper can also foster the community to investigate these challenges with further studies.

Besides industry adoption, we also plan to increase the adoption within the software product line and variability modelling community and beyond to the general software engineering community. Visibility at the main events as well as demonstrators and examples, together with guidance material, are essential to achieve that. Including further researchers in the MODEVAR initiative can have a snowball effect, if the researchers also start to use UVL in their collaborations with academia and industry as well as for teaching.

A key challenge to address is a more in-depth evaluation of UVL's simplicity, efficiency, and applicability in real-world scenarios. We show in this paper some indicators about these aspects, but a formal evaluation is still missing.

The area of teaching is yet another big opportunity to increase the adoption of UVL. The already existing documentation, example models, and UVL playground[16] are a very good starting point, however, we also need to prepare specific material for teaching UVL. MODEVAR community members needs to start using UVL in their teaching and report experiences.

[16] https://universal-variability-language.github.io/

8. Conclusions

During the last decades, feature modelling and analysis have been one of the main research topics in software product line engineering. UVL is a new language for textually modelling variability informed by a participatory process within the software product line community. The language is being used in different existing tools and is a proposal for the community to adopt in the future. A single language cannot fit all the variability needs of different scenarios, unless the language gets more and more complex to cover more needs. That is why UVL is designed using different language levels and includes extension mechanisms. As a result, UVL consists of a simple core language and allows users to extend the language to their specific needs. UVL then allows users to support all UVL models of other levels as well. Its simplicity allows information sharing among researchers, and we envision that it can be used in other scenarios, not only in software product lines.

Although the presented version of the language is stable, and we envision no major changes in the future, if UVL is widely adopted as we pursue, many challenges and research opportunities will appear. We plan to maintain and eventually enlarge a consortium of researchers who discuss the progress of the language and agree on the language's evolution every year. We plan to enlarge and maintain the tool chain supporting UVL such as modelling (Heß et al., 2024; Sundermann et al., 2021b), analysis (Galindo et al., 2023) or sharing (Romero et al., 2021) capabilities. With UVL, variability modelling can be adopted in many application domains and can be a central point for information sharing, tools integration, and variability modelling learning.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

Partially funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – SFB 1608 – 501798263. It was also partially supported by FEDER/Ministry of Science, Innovation and Universities/Junta de Andalucía/State Research Agency/CDTI with the following grants: *Data-pl*(PID2022-138486OB-I00), TASOVA PLUS research network (RED2022-134337-T) and MIDAS (IDI-20230256). The financial support by the Austrian Federal Ministry for Digital and Economic Affairs, the National Foundation for Research, Technology and Development, and the Christian Doppler Research Association is gratefully acknowledged. The authors thank Sebastian Krieter for insightful discussions on the language design of UVL and his contributions to the tools presented. Special thanks go to David Romero, who implemented UVLHub as a tool on top of UVL and helped us to detect some issues in the language. In addition, we would like to give a big thanks to all researchers who have contributed to the MODEVAR initiative throughout these years.

Material

All the source code and data can be downloaded and executed from the following repository: https://github.com/Universal-Variability-Language.

References

Abbasi, Ebrahim Khalil, Hubaux, Arnaud, Acher, Mathieu, Boucher, Quentin, Heymans, Patrick, 2013. The anatomy of a sales configurator: An empirical study of 111 cases. In: Proceedings of the 25th International Conference on Advanced Information Systems Engineering. Springer, pp. 162–177.

Acher, Mathieu, Collet, Philippe, Benavides, David, Rabiser, Rick, 2020. Third international workshop on languages for modelling variability (MODEVAR@ SPLC 2020). In: Proceedings of the 24th ACM Conference on Systems and Software Product Line: Volume a-Volume a. 1–1.

Acher, Mathieu, Collet, Philippe, Lahire, Philippe, France, Robert B., 2009. Composing feature models. In: Proc. Int'l Conf. on Software Language Engineering. SLE, Springer, Denver, CO, USA, pp. 62–81.

Acher, Mathieu, Collet, Philippe, Lahire, Philippe, France, Robert B., 2013. Familiar: A Domain-Specific Language for Large Scale Management of Feature Models. Sci. Comput. Program. 78 (6), 657–681.

Agarwal, Prankur, Feichtinger, Kevin, Schmid, Klaus, Eichelberger, Holger, Rabiser, Rick, 2024. On the challenges of transforming UVL to IVML. http://dx.doi.org/10.48550/arXiv.2403.01952, arXiv:2403.01952 [cs].

Alférez, Mauricio, Acher, Mathieu, Galindo, José Angel, Baudry, Benoit, Benavides, David, 2019. Modeling variability in the video domain: language and experience report. Softw. Qual. J. 27 (1), 307–347. http://dx.doi.org/10.1007/s11219-017-9400-8.

Apel, Sven, Batory, Don, Kästner, Christian, Saake, Gunter, 2013. Feature-Oriented Software Product Lines. Springer, Berlin, Heidelberg, http://dx.doi.org/10.1007/978-3-642-37521-7.

Batory, Don, 2005. Feature models, grammars, and propositional formulas. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC, Springer, Berlin, Heidelberg, pp. 7–20. http://dx.doi.org/10.1007/11554844_3.

Batory, Don, 2020a. Automated Software Design Volume 1. Lulu Press.

Becker, Martin, Rabiser, Rick, Botterweck, Goetz, 2024. Not quite there yet: Remaining challenges in systems and software product line engineering as perceived by industry practitioners. In: Proceedings of the 28th ACM International Systems and Software Product Line Conference. ACM.

Beek, Maurice H. ter, Schmid, Klaus, Eichelberger, Holger, 2019. Textual variability modeling languages: An overview and considerations. In: Proc. Int'l Workshop on Languages for Modelling Variability. MODEVAR, ACM, New York, NY, USA, pp. 151–157. http://dx.doi.org/10.1145/3307630.3342398.

Benavides, David, Rabiser, Rick, Batory, Don, Acher, Mathieu, 2019. First International workshop on languages for modelling variability (MODEVAR 2019). In: Proc. Int'l Systems and Software Product Line Conf.. SPLC, 323–323.

Benavides, David, Segura, Sergio, Ruiz-Cortés, Antonio, 2010. Automated analysis of feature models 20 years later: A literature review. Inf. Syst. 35 (6), 615–708.

Berger, Thorsten, Collet, Philippe, 2019. Usage scenarios for a common feature modeling language. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC, ACM, New York, NY, USA, pp. 174–181.

Berger, Thorsten, Rublack, Ralf, Nair, Divya, Atlee, Joanne M., Becker, Martin, Czarnecki, Krzysztof, Wąsowski, Andrzej, 2013. A survey of variability modeling in industrial practice. In: Proc. Int'l Workshop on Variability Modelling of Software-Intensive Systems. VaMoS, ACM, New York, NY, USA, pp. 7:1–7:8. http://dx.doi.org/10.1145/2430502.2430513.

Berger, Thorsten, Steghöfer, Jan-Philipp, Ziadi, Tewfik, Robin, Jacques, Martinez, Jabier, 2020. The state of adoption and the challenges of systematic variability management in industry. Empir. Softw. Eng. 25 (3), 1755–1797.

Cashman, Mikaela, Firestone, Justin, Cohen, Myra B., Thianniwet, Thammasak, Niu, Wei, 2019. DNA as features: Organic software product lines. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC, ACM, New York, NY, USA, pp. 108–118. http://dx.doi.org/10.1145/3336294.3336298.

Classen, Andreas, Boucher, Quentin, Heymans, Patrick, 2011. A text-based approach to feature modelling: Syntax and semantics of TVL. Sci. Comput. Program. 76 (12), 1130–1143, Special Issue on Software Evolution, Adaptability and Variability.

Czarnecki, Krzysztof, Helsen, Simon, Eisenecker, Ulrich, 2005. Formalizing cardinality-based feature models and their specialization. Software Process: Improv. Pract. 10, 7–29.

Czarnecki, Krzysztof, Helsen, Simon, Eisenecker, Ulrich, 2005b. Staged configuration through specialization and multi-level configuration of feature models. Software Process: Improv. Pract. 10 (2), 143–169.

Czarnecki, Krzysztof, Kim, Chang Hwan Peter, 2005. Cardinality-based feature modeling and constraints: A progress report. In: Proc. Int'l Workshop on Software Factories. SF, University of Waterloo, pp. 16–20.

Czarnecki, Krzysztof, Wąsowski, Andrzej, 2007. Feature diagrams and logics: There and back again. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC, IEEE, Washington, DC, USA, pp. 23–34.

de Moura, Leonardo, Bjørner, Nikolaj, 2008. Z3: An efficient SMT solver. In: Ramakrishnan, C.R., Rehof, Jakob (Eds.), Tools and Algorithms for the Construction and Analysis of Systems. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 337–340.

De Moura, Leonardo, Bjørner, Nikolaj, 2011. Satisfiability modulo theories: Introduction and applications. Commun. ACM 54 (9), 69–77.

Dhungana, Deepak, Grünbacher, Paul, Rabiser, Rick, 2011. The DOPLER meta-tool for decision-oriented variability modeling: A multiple case study. Autom. Softw. Eng. 18 (1), 77–114.

Eichelberger, Holger, Schmid, Klaus, 2015. Mapping the design space of textual variability modeling languages: A refined analysis. Int'l J. Software Tools Technol. Transf. (STTT) 17 (5), 559–584. http://dx.doi.org/10.1007/s10009-014-0362-x.

El-Sharkawy, Sascha, Krafczyk, Adam, Schmid, Klaus, 2015. Analysing the kconfig semantics and its analysis tools. In: Proc. Int'l Conf. on Generative Programming: Concepts & Experiences. GPCE, ACM, New York, NY, USA, pp. 45–54. http://dx.doi.org/10.1145/2814204.2814222.

Fadhlillah, Hafiyyan Sayyid, Feichtinger, Kevin, Bauer, Philipp, Kutsia, Elene, Rabiser, Rick, 2022. V4rdiac: tooling for multidisciplinary delta-oriented variability management in cyber-physical production systems. In: Proceedings of the 26th ACM International Systems and Software Product Line Conference - Volume B. SPLC '22, Association for Computing Machinery, New York, NY, USA, pp. 34–37. http://dx.doi.org/10.1145/3503229.3547028.

Fadhlillah, Hafiyyan Sayyid, Rabiser, Rick, 2024. Towards a product configuration representation for the universal variability language. In: Proceedings of the 28th ACM International Systems and Software Product Line Conference (Dommeldange, Luxembourg). SPLC '24, Association for Computing Machinery, New York, NY, USA, pp. 50–54. http://dx.doi.org/10.1145/3646548.3676544.

Feichtinger, Kevin, Galasso-Carbonnel, Jessie, 2024. Seventh international workshop on languages for modelling variability (MODEVAR@SPLC 2024). In: Proceedings of the 28th ACM International Systems and Software Product Line Conference (Dommeldange, Luxembourg). SPLC '24, Association for Computing Machinery, New York, NY, USA, p. 224. http://dx.doi.org/10.1145/3646548.3677006.

Feichtinger, Kevin, Meixner, Kristof, Biffl, Stefan, Rabiser, Rick, 2022. Evolution support for custom variability artifacts using feature models: A study in the cyber-physical production systems domain. In: Perrouin, Gilles, Moha, Naouel, Seriai, Abdelhak-Djamel (Eds.), Reuse and Software Quality. Springer International Publishing, Cham, pp. 79–84.

Feichtinger, Kevin, Rabiser, Rick, 2020a. Towards transforming variability models: Usage scenarios, required capabilities and challenges. In: Proc. Int'l Workshop on Languages for Modelling Variability. MODEVAR (Montreal, QC, Canada), In: SPLC '20, ACM, New York, NY, USA, pp. 44–51. http://dx.doi.org/10.1145/3382026.3425768.

Feichtinger, Kevin, Rabiser, Rick, 2020b. Variability model transformations: Towards unifying variability modeling. In: 46th Euromicro Conference on Software Engineering and Advanced Applications. IEEE, Portoroz, Slovenia.

Feichtinger, Kevin, Stöbich, Johann, Romano, Dario, Rabiser, Rick, 2021. TRAVART: An approach for transforming variability models. In: Proc. Int'l Working Conf. on Variability Modelling of Software-Intensive Systems. VaMoS (Krems, Austria), ACM, New York, NY, USA, pp. 8:1–8:10. http://dx.doi.org/10.1145/3442391.3442400.

Felfernig, Alexander, Falkner, Andreas, Benavides, David, 2024. Feature Models: AI-Driven Design, Analysis and Applications. Springer Nature, http://dx.doi.org/10.1007/978-3-031-61874-1.

Galindo, José A., Benavides, David, 2020. A python framework for the automated analysis of feature models: A first step to integrate community efforts. In: Proceedings of the 24th Acm International Systems and Software Product Line Conference-Volume B. pp. 52–55.

Galindo, José A., Benavides, David, Trinidad, Pablo, Gutiérrez-Fernández, Antonio-Manuel, Ruiz-Cortés, Antonio, 2019. Automated analysis of feature models: Quo Vadis? Computing 101 (5), 387–433. http://dx.doi.org/10.1007/s00607-018-0646-1.

Galindo, José A., Horcas, José Miguel, Felfernig, Alexander, Fernández-Amorós, David, Benavides, David, 2023. FLAMA: A collaborative effort to build a new framework for the automated analysis of feature models. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC, ACM, New York, NY, USA, pp. 16–19. http://dx.doi.org/10.1145/3579028.3609008.

Haugen, Øystein, Wąsowski, Andrzej, Czarnecki, Krzysztof, 2012. CVL: Common variability language. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC (Salvador, Brazil), ACM, New York, NY, USA, pp. 266–267. http://dx.doi.org/10.1145/2364412.2364462.

Heradio, Ruben, Fernández-Amorós, David, Galindo, José A., Benavides, David, Batory, Don S., 2022. Uniform and scalable sampling of highly configurable systems. Empir. Softw. Eng. (EMSE) 27 (2), 44. http://dx.doi.org/10.1007/s10664-021-10102-5.

Heß, Tobias, Müller, Tobias, Sundermann, Chico, Thüm, Thomas, 2022a. ddueruem: A wrapper for feature-model analysis tools. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC (Graz, Austria), ACM, New York, NY, USA, pp. 54–57. http://dx.doi.org/10.1145/3503229.3547032.

Heß, Tobias, Müller, Tobias, Sundermann, Chico, Thüm, Thomas, 2022b. Ddueruem: a wrapper for feature-model analysis tools. In: Proceedings of the 26th ACM International Systems and Software Product Line Conference - Volume B. SPLC '22, Association for Computing Machinery, New York, NY, USA, pp. 54–57. http://dx.doi.org/10.1145/3503229.3547032.

Heß, Tobias, Ostheimer, Lukas, Betz, Tobias, Karrer, Simon, Schmidt, Tim Jannik, Coquet, Pierre, Semmler, Sean, Thüm, Thomas, 2024. variability.dev: Towards an online toolbox for feature modeling. In: Proc. Int'l Workshop on Languages for Modelling Variability. MODEVAR (Bern, Switzerland).

Horcas, Jose M., Galindo, Jose A., Pinto, Mónica, Fuentes, Lidia, Benavides, David, 2022a. FM fact label: a configurable and interactive visualization of feature model characterizations. In: Proceedings of the 26th ACM International Systems and Software Product Line Conference - Volume B. SPLC '22, Association for Computing Machinery, New York, NY, USA, pp. 42–45. http://dx.doi.org/10.1145/3503229.3547025.

Horcas, Jose-Miguel, Villota, Angela, Benavides, David, Collet, Philippe, 2022b. Fifth international workshop on languages for modelling variability (MODEVAR@ SPLC 2022). In: Proceedings of the 26th ACM International Systems and Software Product Line Conference-Volume A. 264–264.

ISO/IEC 26558:2017(en), 2017. Software and Systems Engineering — Methods and Tools for Variability Modelling in Software and Systems Product Line. Standard 2017, International Organization for Standardization/International Electrotechnical Commission, Geneva, CH, https://www.iso.org/obp/ui/#iso:std:iso-iec:26558:ed-1:v1:en.

Jussien, Narendra, Rochart, Guillaume, Lorca, Xavier, 2008. Choco: An open source java constraint programming library. In: Proc. Workshop on Open-Source Software for Integer and Contraint Programming. OSSICP, CCSD-HAL, Lyon, France.

Kang, Kyo C., Cohen, Sholom G., Hess, James A., Novak, William E., Peterson, A. Spencer, 1990. Feature-Oriented Domain Analysis (FODA) Feasibility Study. Technical Report CMU/SEI-90-TR-21, Software Engineering Institute, Pittsburgh, PA, USA.

Kästner, Christian, Dreiling, Alexander, Ostermann, Klaus, 2013. Variability mining: Consistent semi-automatic detection of product-line features. IEEE Trans. Softw. Eng. 40 (1), 67–82.

Kästner, Christian, Thüm, Thomas, Saake, Gunter, Feigenspan, Janet, Leich, Thomas, Wielgorz, Fabian, Apel, Sven, 2009. FeatureIDE: A tool framework for feature-oriented software development. In: Proc. Int'l Conf. on Software Engineering. ICSE (Vancouver, Canada), IEEE, Washington, DC, USA, pp. 611–614. http://dx.doi.org/10.1109/ICSE.2009.5070568, Formal demonstration paper.

Knüppel, Alexander, Thüm, Thomas, Mennicke, Stephan, Meinicke, Jens, Schaefer, Ina, 2017. Is there a mismatch between real-world feature models and product-line research? In: Proc. Europ. Software Engineering Conf./Foundations of Software Engineering. ESEC/FSE (Paderborn, Germany), ACM, New York, NY, USA, pp. 291–302. http://dx.doi.org/10.1145/3106237.3106252.

Krieter, Sebastian, Feichtinger, Kevin, Galindo, José A., Benavides, David, Rabiser, Rick, Sundermann, Chico, Thüm, Thomas, 2023. Second tutorial on the universal variability language. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC (Tokyo, Japan), ACM, New York, NY, USA, p. 273. http://dx.doi.org/10.1145/3579027.3609002.

Kübler, Andreas, Zengler, Christoph, Küchlin, Wolfgang, 2010. Model counting in product configuration. In: Proc. Int'l Workshop on Logics for Component Configuration. LoCoCo (Edinburgh, UK), Open Publishing Association, Waterloo, Australia, pp. 44–53. http://dx.doi.org/10.4204/EPTCS.29.5.

Kuiter, Elias, Krieter, Sebastian, Sundermann, Chico, Thüm, Thomas, Saake, Gunter, 2022. Tseitin or not Tseitin? The impact of CNF transformations on feature-model analyses. In: Proc. Int'l Conf. on Automated Software Engineering. ASE (Rochester, MI, USA), ACM, New York, NY, USA, pp. 110:1–110:13. http://dx.doi.org/10.1145/3551349.3556938.

Le, Viet-Man, Tran, Thi Ngoc Trang, Stettinger, Martin, Weißl, Lisa, Felfernig, Alexander, Atas, Müslüm, Erdeniz, Seda Polat, Popescu, Andrei, 2021. Counteracting Exam Cheating by Leveraging Configuration and Recommendation Techniques. In: ConfWS. pp. 73–80.

Liang, Jia Hui, Ganesh, Vijay, Czarnecki, Krzysztof, Raman, Venkatesh, 2015. SAT-Based analysis of large real-world feature models is easy. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC, Springer, Berlin, Heidelberg, pp. 91–100.

Loth, Jacob, Sundermann, Chico, Schrull, Tobias, Brugger, Thilo, Rieg, Felix, Thüm, Thomas, 2023. UVLS: A language server protocol for UVL. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC (Tokyo, Japan), New York, NY, USA, pp. 43–46. http://dx.doi.org/10.1145/3579028.3609014.

Lotufo, Rafael, She, Steven, Berger, Thorsten, Czarnecki, Krzysztof, Wąsowski, Andrzej, 2010. Evolution of the Linux Kernel variability model. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC (Jeju Island, South Korea), Springer, Berlin, Heidelberg, pp. 136–150.

Márquez, Germán, Galindo, José A., Varela-Vaca, Ángel Jesús, López, María Teresa Gómez, Benavides, David, 2022. Advisory: vulnerability analysis in software development project dependencies. In: Proceedings of the 26th ACM International Systems and Software Product Line Conference - Volume B. SPLC '22, Association for Computing Machinery, New York, NY, USA, pp. 99–102. http://dx.doi.org/10.1145/3503229.3547058.

Matos, Simone Nasser, Fernandes, Clovis Torres, 2006. Early definition of frozen and hot spots in the development of domain frameworks. In: Fourteenth ACM SIGSOFT Symposium on Foundations of Software Engineering.

Meinicke, Jens, Thüm, Thomas, Schröter, Reimar, Benduhn, Fabian, Leich, Thomas, Saake, Gunter, 2017. Mastering Software Variability With FeatureIDE. Springer, Berlin, Heidelberg, http://dx.doi.org/10.1007/978-3-319-61443-4.

Meixner, Kristof, Feichtinger, Kevin, Fadhlillah, Hafiyyan Sayyid, Greiner, Sandra, Marcher, Hannes, Rabiser, Rick, Biffl, Stefan, 2024. Variability modeling of products, processes, and resources in cyber–physical production systems engineering. J. Syst. Softw. 211, 112007. http://dx.doi.org/10.1016/j.jss.2024.112007, https://www.sciencedirect.com/science/article/pii/S0164121224000505.

Mendonça, Marcílio, Branco, Moises, Cowan, Donald, 2009a. S.P.L.O.T.: Software product lines online tools. In: Proc. Conf. on Object-Oriented Programming, Systems, Languages and Applications. OOPSLA, ACM, New York, NY, USA, pp. 761–762.

Mendonça, Marcílio, Wąsowski, Andrzej, Czarnecki, Krzysztof, 2009b. SAT-Based analysis of feature models is easy. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC (San Francisco, California), Software Engineering Institute, Pittsburgh, PA, USA, pp. 231–240.

Munoz, Daniel-Jesus, Oh, Jeho, Pinto, Mónica, Fuentes, Lidia, Batory, Don, 2019. Uniform random sampling product configurations of feature models that have numerical features. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC (Paris, France), ACM, New York, NY, USA, pp. 289–301. http://dx.doi.org/10.1145/3336294.3336297.

Munoz, Daniel-Jesus, Pinto, Mónica, Fuentes, Lidia, Batory, Don, 2023. Transforming numerical feature models into propositional formulas and the universal variability language. J. Syst. Softw. 204, 111770. http://dx.doi.org/10.1016/j.jss.2023.111770, https://www.sciencedirect.com/science/article/pii/S0164121223001656.

Nesić, Damir, Krüger, Jacob, Stănciulescu, Ştefan, Berger, Thorsten, 2019. Principles of feature modeling. In: Proceedings of the 2019 27th ACM Joint Meeting of European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, pp. 62–73.

Oh, Jeho, Batory, Don, Heradio, Rubén, 2023. Finding near-optimal configurations in colossal spaces with statistical guarantees. ACM Trans. Softw. Eng. Methodol. 33 (1), 1–36.

Parr, Terence, 2013. The Definitive ANTLR 4 Reference. The Pragmatic Bookshelf, pp. 1–326.

pure::systems, 2017. pure::variants. Website: http://www.pure-systems.com/products/pure-variants-9.html. (Accessed: 2017-05-10).

Rabiser, Rick, 2019. Feature modeling vs. Decision modeling: History, comparison and perspectives. In: Proceedings of the 23rd International Systems and Software Product Line Conference - Volume B (Paris, France). SPLC '19, Association for Computing Machinery, New York, NY, USA, pp. 134–136. http://dx.doi.org/10.1145/3307630.3342399.

Rabiser, Rick, 2024. Industry adoption of UVL: What we will need. In: Proceedings of the 28th ACM International Systems and Software Product Line Conference Vol. 2. ACM.

Riebisch, Matthias, Böllert, Kai, Streitferdt, Detlef, Philippow, Ilka, 2002. Extending feature diagrams with UML multiplicities. In: Proc. World Conf. on Integrated Design and Process Technology. IDPT.

Rodas-Silva, Jorge, Galindo, José A, García-Gutiérrez, Jorge, Benavides, David, 2019. Selection of software product line implementation components using recommender systems: An application to wordpress. IEEE Access 7, 69226–69245.

Romano, Dario, Feichtinger, Kevin, Beuche, Danilo, Ryssel, Uwe, Rabiser, Rick, 2022. Bridging the gap between academia and industry: transforming the universal variability language to pure::variants and back. In: Proceedings of the 26th ACM International Systems and Software Product Line Conference - Volume B (Graz, Austria). SPLC '22, Association for Computing Machinery, New York, NY, USA, pp. 123–131. http://dx.doi.org/10.1145/3503229.3547056.

Romero, David, Galindo, José A., Horcas, José Miguel, Benavides, David, 2021. A first prototype of a new repository for feature model exchange and knowledge sharing. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC, ACM, New York, NY, USA, pp. 80–85. http://dx.doi.org/10.1145/3461002.3473949.

Romero-Organvidez, David, Galindo, Jose A., Benavides, David, 2024a. UVL sentinel: a tool for parsing and syntactic correction of UVL datasets. https://arxiv.org/abs/2403.18482. arXiv:2403.18482 [cs.SE].

Romero-Organvidez, David, Galindo, José A., Sundermann, Chico, Horcas, Jose-Miguel, Benavides, David, 2024b. UVLHub: A feature model data repository using UVL and open science principles. J. Syst. Softw. 112150. http://dx.doi.org/10.1016/j.jss.2024.112150, https://www.sciencedirect.com/science/article/pii/S016412122400195X.

Rothberg, Valentin, Dintzner, Nicolas, Ziegler, Andreas, Lohmann, Daniel, 2016. Feature models in Linux: From symbols to semantics. In: Proc. Int'l Workshop on Variability Modelling of Software-Intensive Systems. VaMoS (Salvador, Brazil), ACM, New York, NY, USA, pp. 65–72. http://dx.doi.org/10.1145/2866614.2866624.

Schobbens, Pierre-Yves, Heymans, Patrick, Trigaux, Jean-Christophe, Bontemps, Yves, 2007. Generic semantics of feature diagrams. Computer Netw. 51 (2), 456–479.

Sundermann, Chico, Feichtinger, Kevin, Engelhardt, Dominik, Rabiser, Rick, Thüm, Thomas, 2021a. Yet another textual variability language? A community effort towards a unified language. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC (Leicester, UK), ACM, New York, NY, USA, pp. 136–147. http://dx.doi.org/10.1145/3461001.3471145.

Sundermann, Chico, Feichtinger, Kevin, Galindo, José A., Benavides, David, Rabiser, Rick, Krieter, Sebastian, Thüm, Thomas, 2022. Tutorial on the universal variability language. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC (Graz, Austria), ACM, New York, NY, USA, p. 260:1. http://dx.doi.org/10.1145/3546932.3547024.

Sundermann, Chico, Heß, Tobias, Engelhardt, Dominik, Arens, Rahel, Herschel, Johannes, Jedelhauser, Kevin, Jutz, Benedikt, Krieter, Sebastian, Schaefer, Ina, 2021b. Integration of UVL in FeatureIDE. In: Proc. Int'l Workshop on Languages for Modelling Variability. MODEVAR (Leicester, UK), ACM, New York, NY, USA, pp. 73–79. http://dx.doi.org/10.1145/3461002.3473940.

Sundermann, Chico, Heß, Tobias, Nieke, Michael, Bittner, Paul Maximilian, Young, Jeffrey M., Thüm, Thomas, Schaefer, Ina, 2023a. Evaluating state-of-the-art #SAT solvers on industrial configuration spaces. Empir. Softw. Eng. 28 (29), 38. http://dx.doi.org/10.1007/s10664-022-10265-9.

Sundermann, Chico, Heß, Tobias, Sundermann, Rahel, Kuiter, Elias, Krieter, Sebastian, Thüm, Thomas, 2024. Generating feature models with UVL's full expressiveness. In: Proceedings of the 28th ACM International Systems and Software Product Line Conference (Dommeldange, Luxembourg). SPLC '24, Association for Computing Machinery, New York, NY, USA, pp. 61–65. http://dx.doi.org/10.1145/3646548.3676602.

Sundermann, Chico, Vill, Stefan, Thüm, Thomas, Feichtinger, Kevin, Agarwal, Prankur, Rabiser, Rick, Galindo, José A., Benavides, David, 2023b. UVLParser: Extending UVL With Language Levels and Conversion Strategies. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC (Tokyo, Japan), ACM, New York, NY, USA, pp. 39–42. http://dx.doi.org/10.1145/3579028.3609013.

Thüm, Thomas, 2020. A BDD for Linux? The knowledge compilation challenge for variability. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC (Montreal, QC, Canada), ACM, New York, NY, USA, 16. http://dx.doi.org/10.1145/3382025.3414943, 6 pages.

Thüm, Thomas, Collet, Philippe, Acher, Mathieu (Eds.), 2021. Fourth International Workshop on Languages for Modelling Variability. MODEVAR@SPLC 2021 (Leicester, UK), ACM, New York, NY, USA, http://dx.doi.org/10.1145/3461001.3473056, 205–205.

Thüm, Thomas, Kästner, Christian, Erdweg, Sebastian, Siegmund, Norbert, 2011. Abstract Features in Feature Modeling. In: Proc. Int'l Systems and Software Product Line Conf.. SPLC (Munich, Germany), IEEE, Washington, DC, USA, pp. 191–200. http://dx.doi.org/10.1109/SPLC.2011.53.

Thüm, Thomas, Seidl, Christoph, Schaefer, Ina, 2019. On language levels for feature modeling notations. In: Proc. Int'l Workshop on Languages for Modelling Variability. MODEVAR (Paris, France), ACM, New York, NY, USA, pp. 158–161. http://dx.doi.org/10.1145/3307630.3342404.

Wąsowski, Andrzej, Berger, Thorsten, 2023. Domain-Specific Languages: Effective modeling, automation, and reuse. Springer.

Zhang, Wei, Zhao, Haiyan, Mei, Hong, 2004. A propositional logic-based method for verification of feature models. In: Proc. Int'l Conf. on Formal Engineering Methods. ICFEM, Springer, Berlin, Heidelberg, pp. 115–130.